



Escola Superior de Tecnologia e Gestão
Instituto Politécnico da Guarda

ALGORITMOS: ÁRVORES BINÁRIAS

2011/2012, A1, S1

PAULO NUNES

AV. DR. FRANCISCO SÁ CARNEIRO, 50 - 6301-559 GUARDA

TELF. 271220120, EXT. 1220, GAB:20

GPS: LATITUDE: 40.5416236730513, LONGITUDE: -7.28243350982666

VOIP: pnunes@ipg.pt, MSN: pnunes@ipg.pt, SKYPE: pnunes.ipg.pt

EMAIL: Mailto:pnunes@ipg.pt, WEB: <http://www.ipg.pt/user/~pnunes/>





ÁRVORES

- ❑ As listas são estruturas que só têm uma dimensão;
- ❑ Quando pretendemos efectuar uma pesquisa temos de percorrer sequencialmente a lista;
- ❑ É difícil usá-las para organizar uma representação hierárquica de objectos;
- ❑ Para superar estes problemas temos a estrutura designada de árvore.

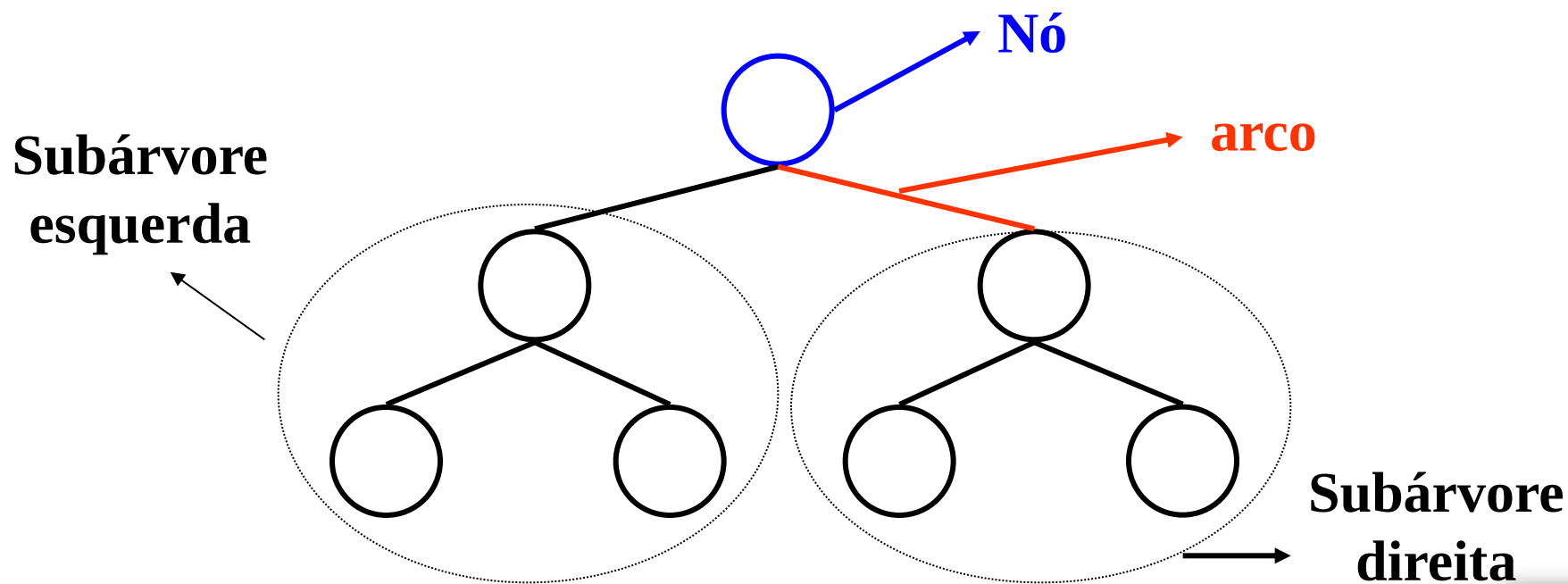


APLICAÇÕES

- ❑ Armazéns de grandes volumes de dados
- ❑ Verificação de frases fazendo parte de linguagens (ex: programas, expressões aritméticas,...)
- ❑ Modelação de sistemas organizacionais (ex: famílias, directórios de um computador, hierarquia de comando de uma empresa,...)

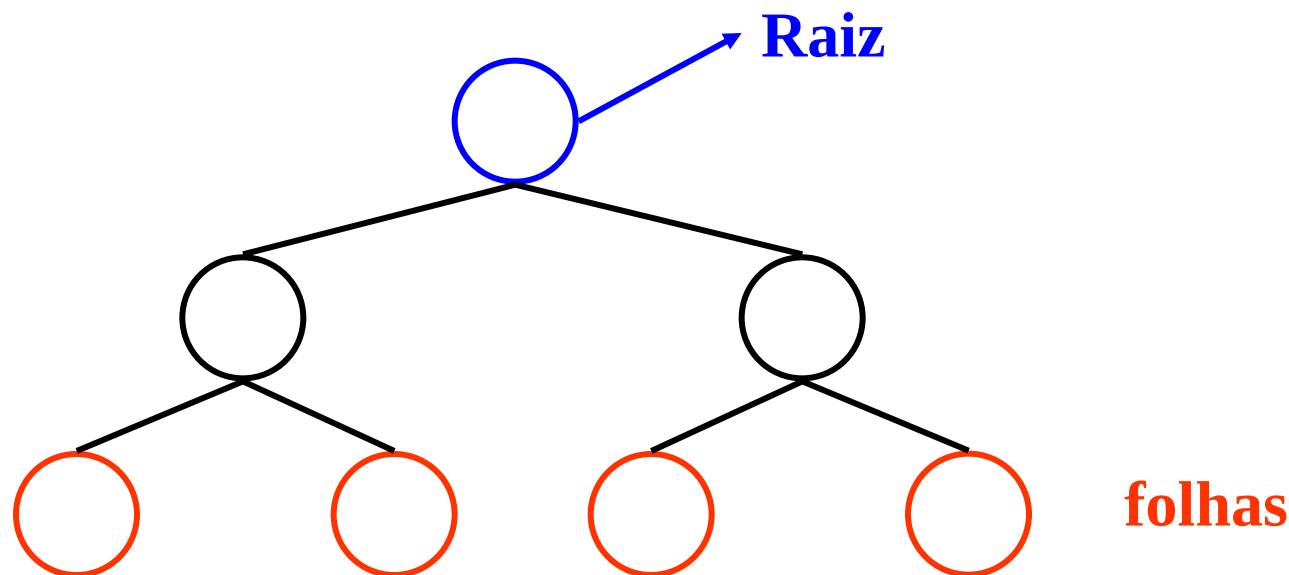
DEFINIÇÃO

- Uma árvore é uma estrutura constituída por **nós** e **arcos**.



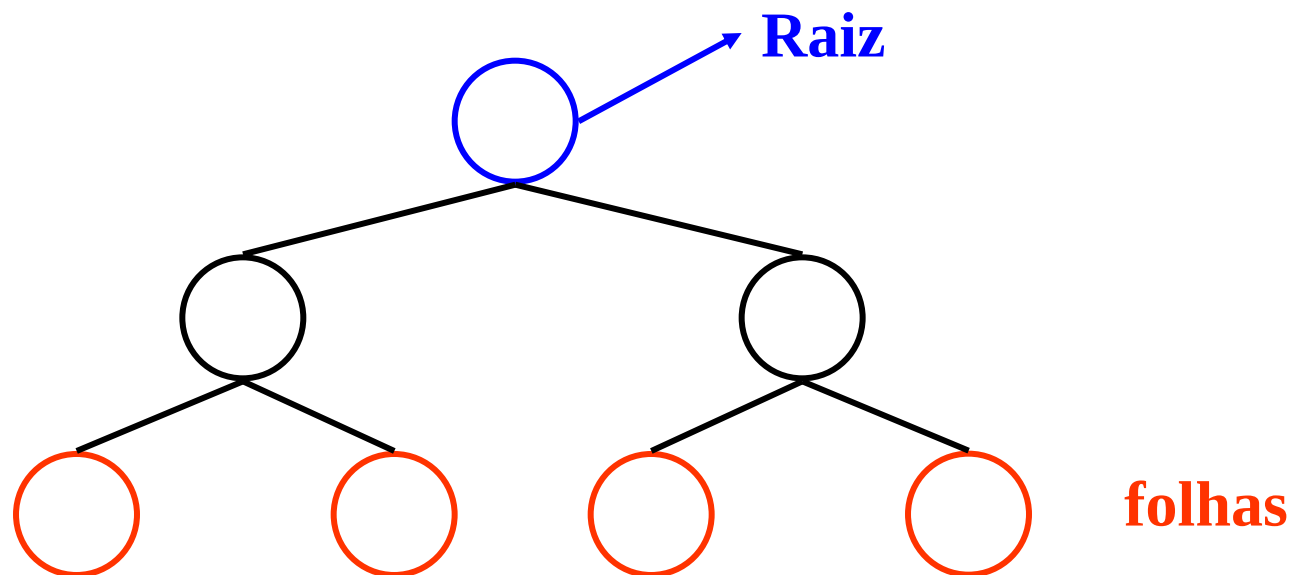
DEFINIÇÃO

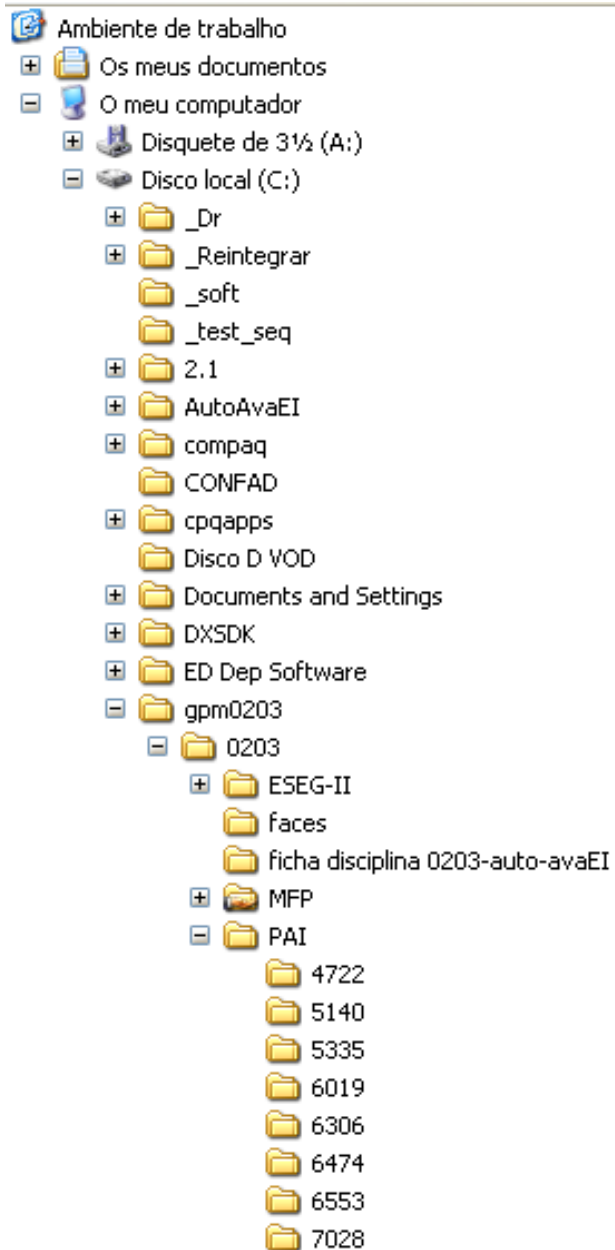
- As árvores são representadas de cima para baixo com a **raiz** no topo e as **folhas** na base.



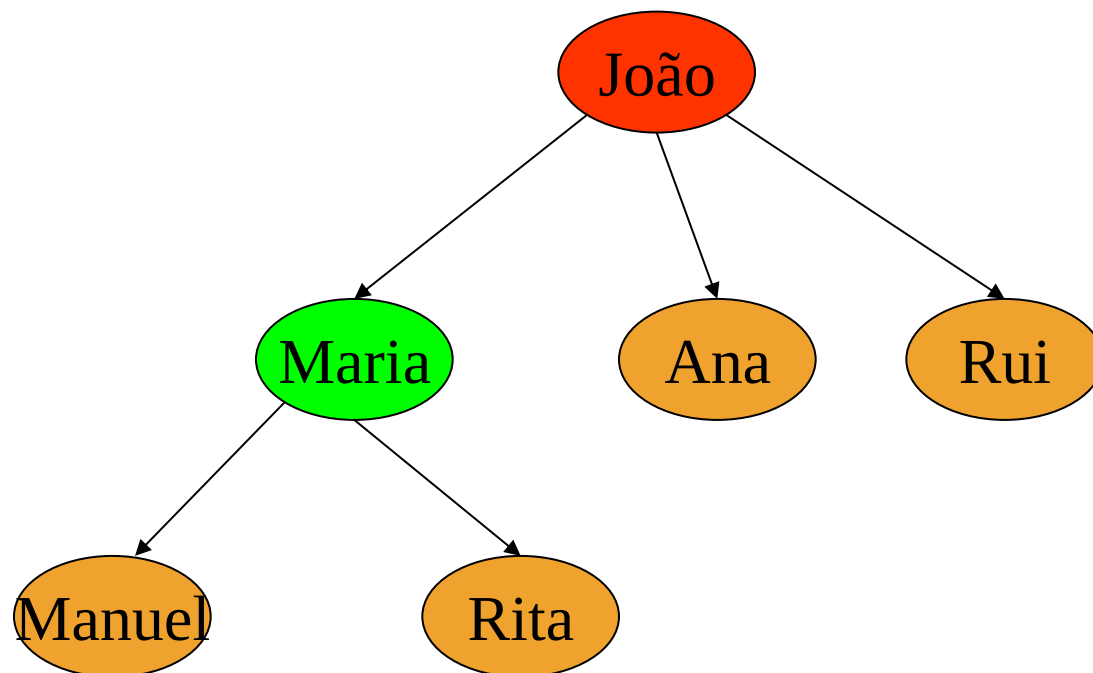
DEFINIÇÃO

- ❑ A **raiz** é um nó que não tem ascendentes.
- ❑ As **folhas**, não têm filhos (descendentes).





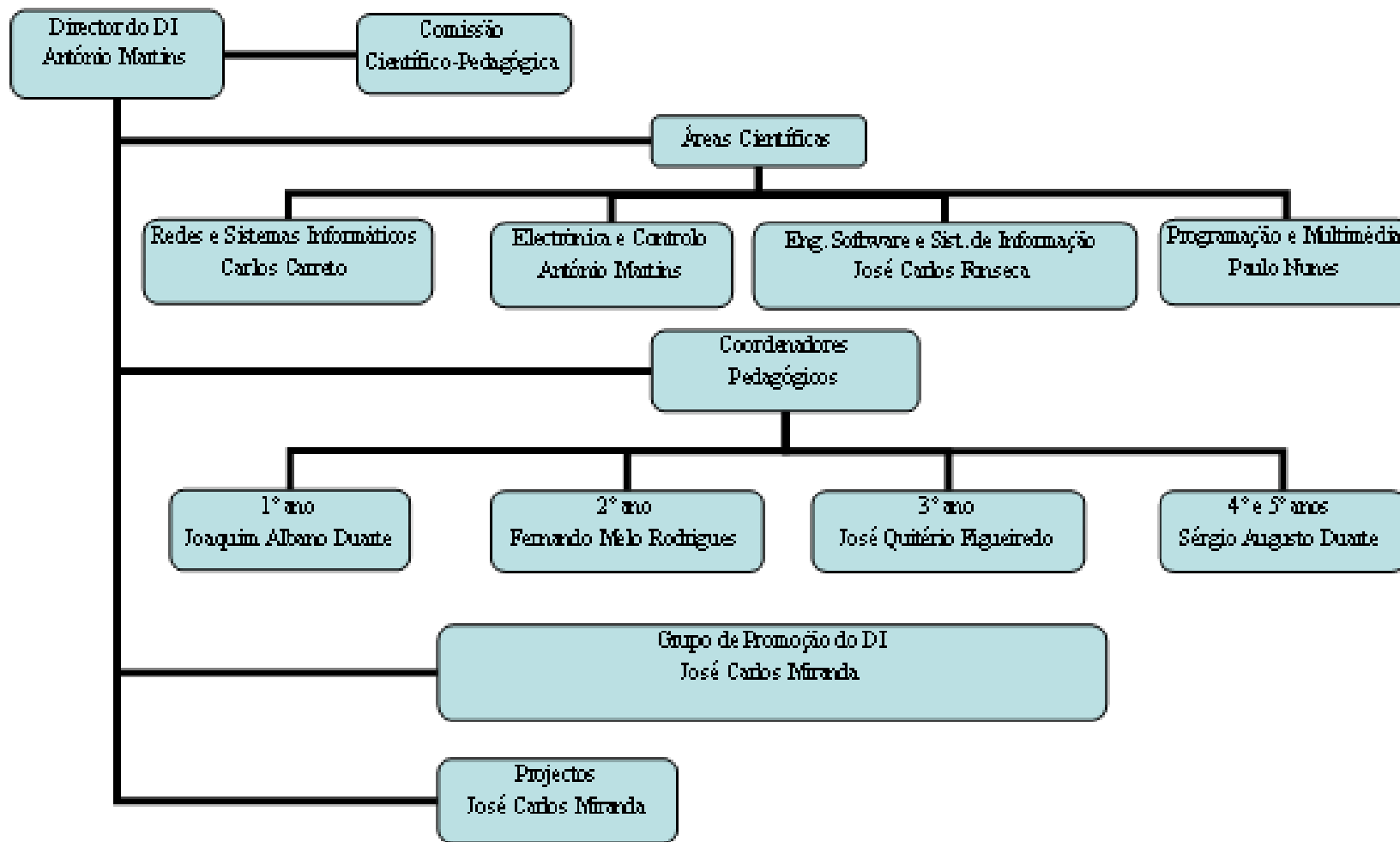
Escola Superior de Tecnologia e Gestão
Instituto Politécnico da Guarda



ORGANIGRAMA DAS ESTRUTURAS DE ACOMPANHAMENTO DO CURSO.



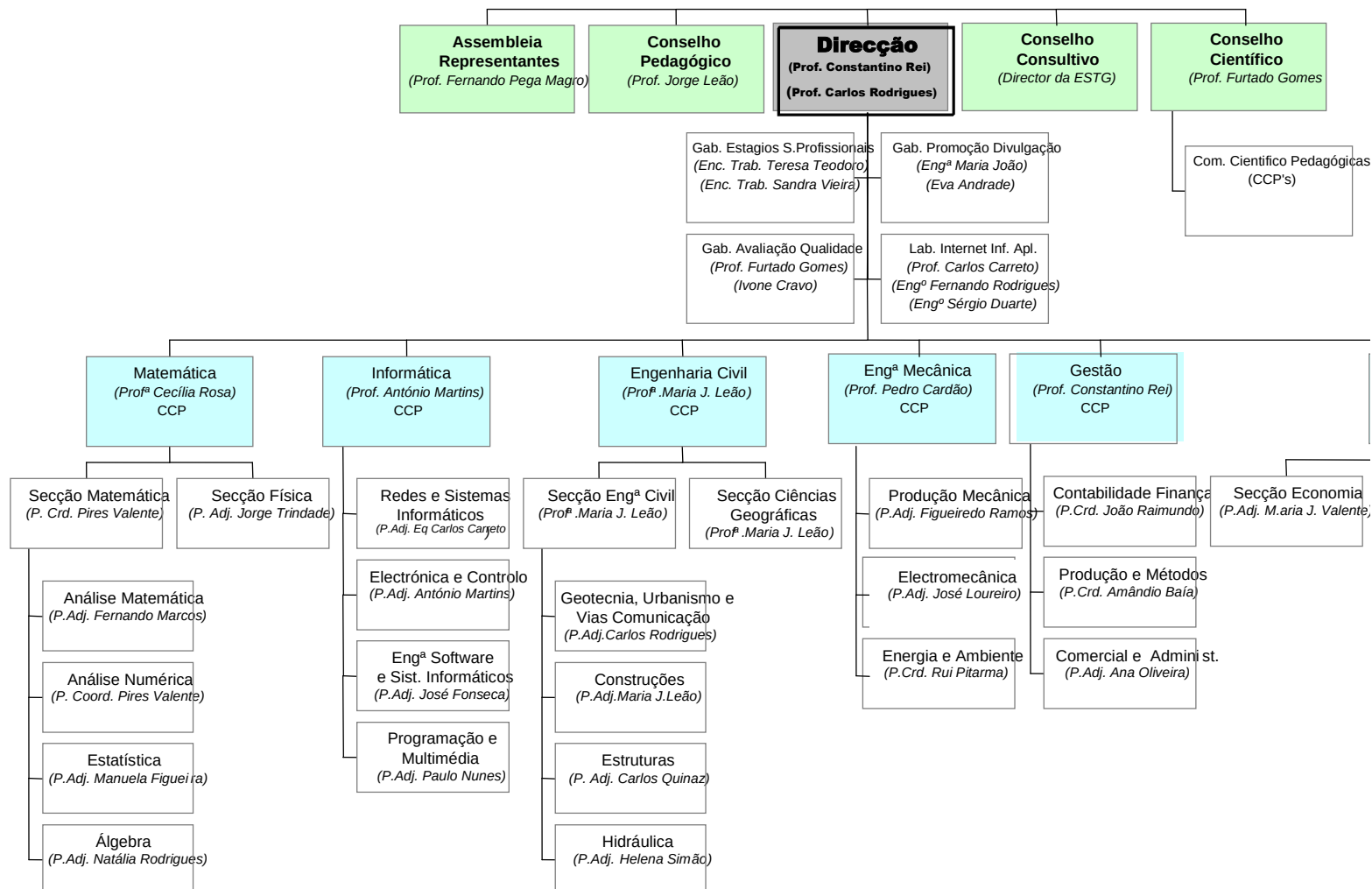
Escola Superior de Tecnologia e Gestão
Instituto Politécnico da Guarda



Fonte: Relatório de autoavaliação de EI 2002/2003.

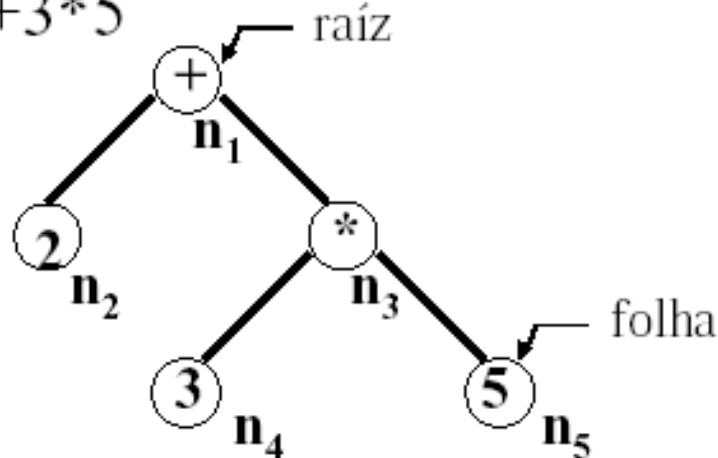


ORGANIZAÇÃO INTERNA DA ESTG



Exemplo A2: expressões aritméticas

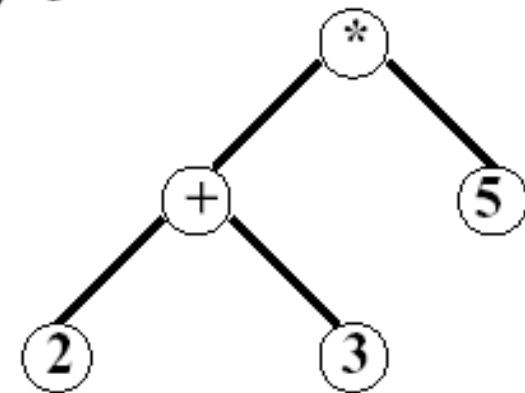
$2+3*5$



$V = \{n_1, n_2, n_3, n_4, n_5\}$

$E = \{(n_1, n_2), (n_1, n_3), (n_3, n_4), (n_3, n_5)\}$

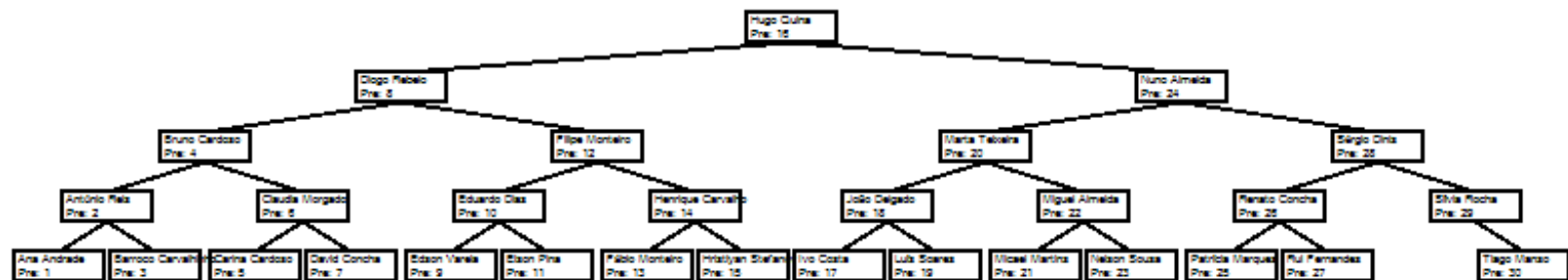
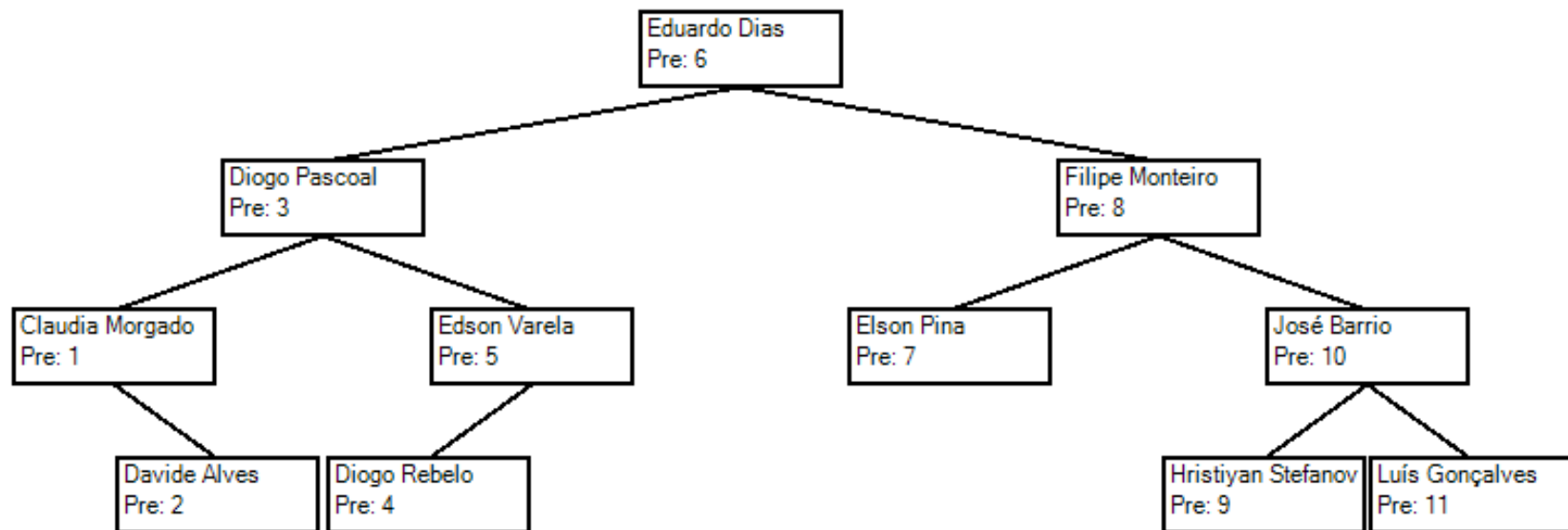
$(2+3)*5$



A árvore é avaliada de forma ascendente ('`bottom-up`'):

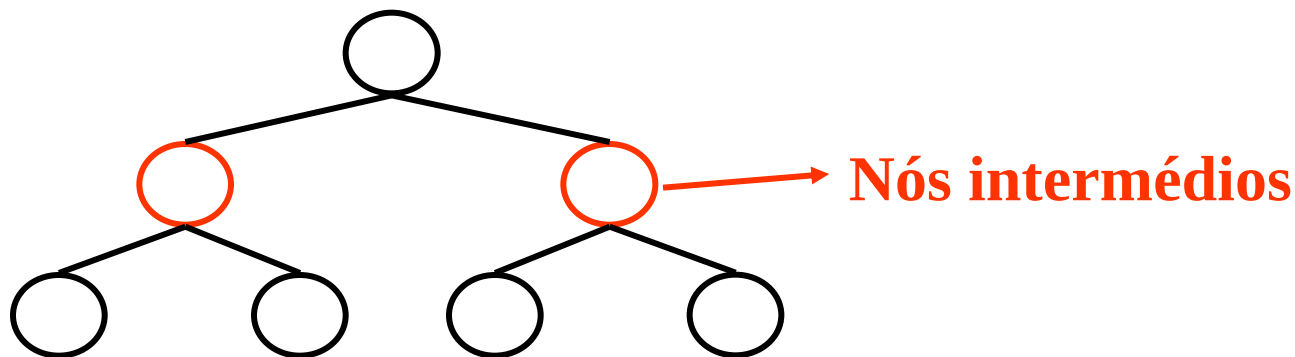
$$2+3*5 = 2+15 = 17$$

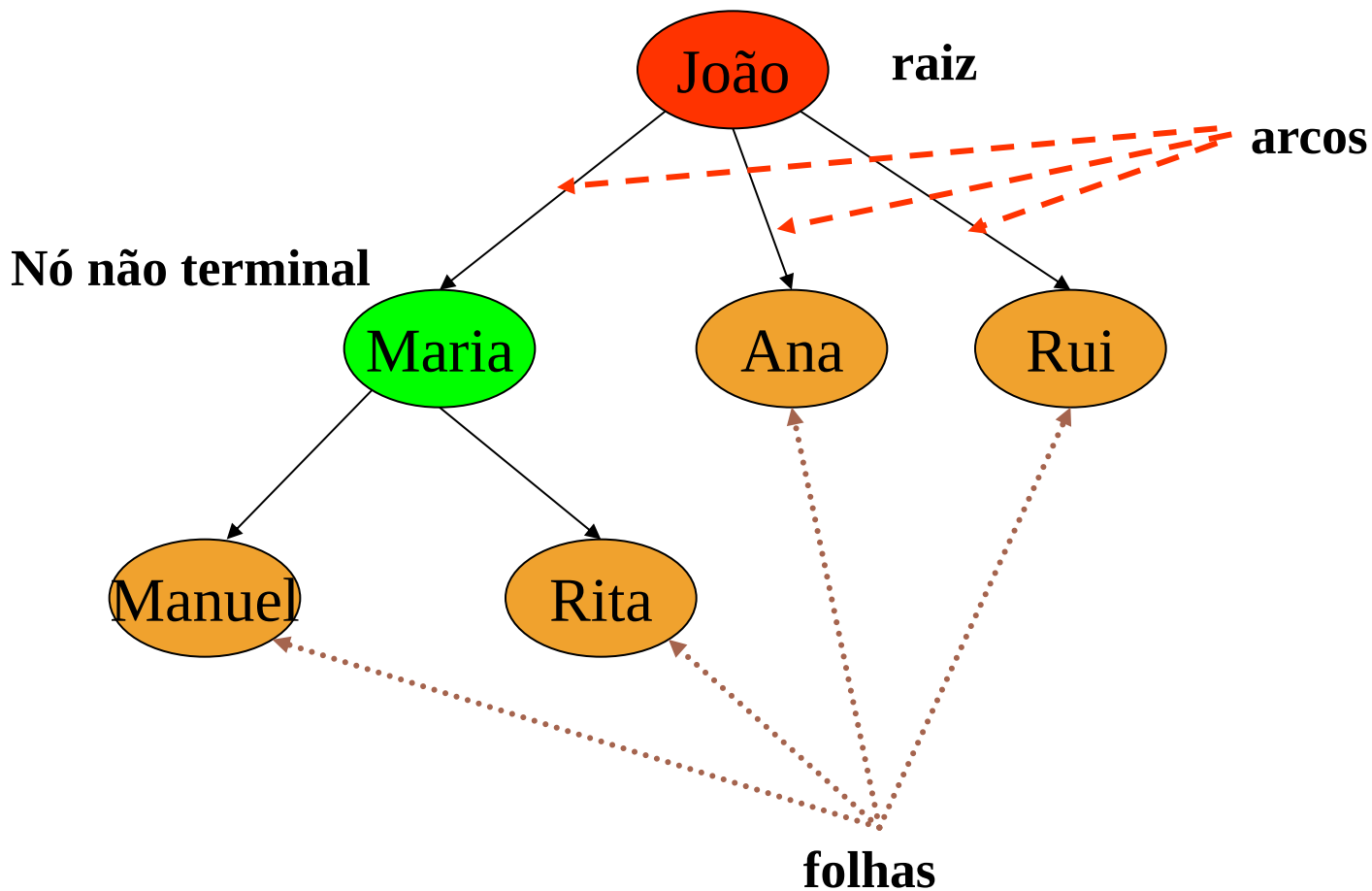
EXEMPLO: ALUNOS AED



CONSIDERAÇÕES

- ❑ Nós com filhos são designados **não-terminais**.
- ❑ Nós não-terminais, distintos da raíz, são designados **intermédios** ou **interiores**.







DEFINIÇÃO

- A árvore é de tipo **K**, sse todos os nós intermédios tiverem **K** filhos.
- Quando $K=2$, a árvore diz-se **binária**.

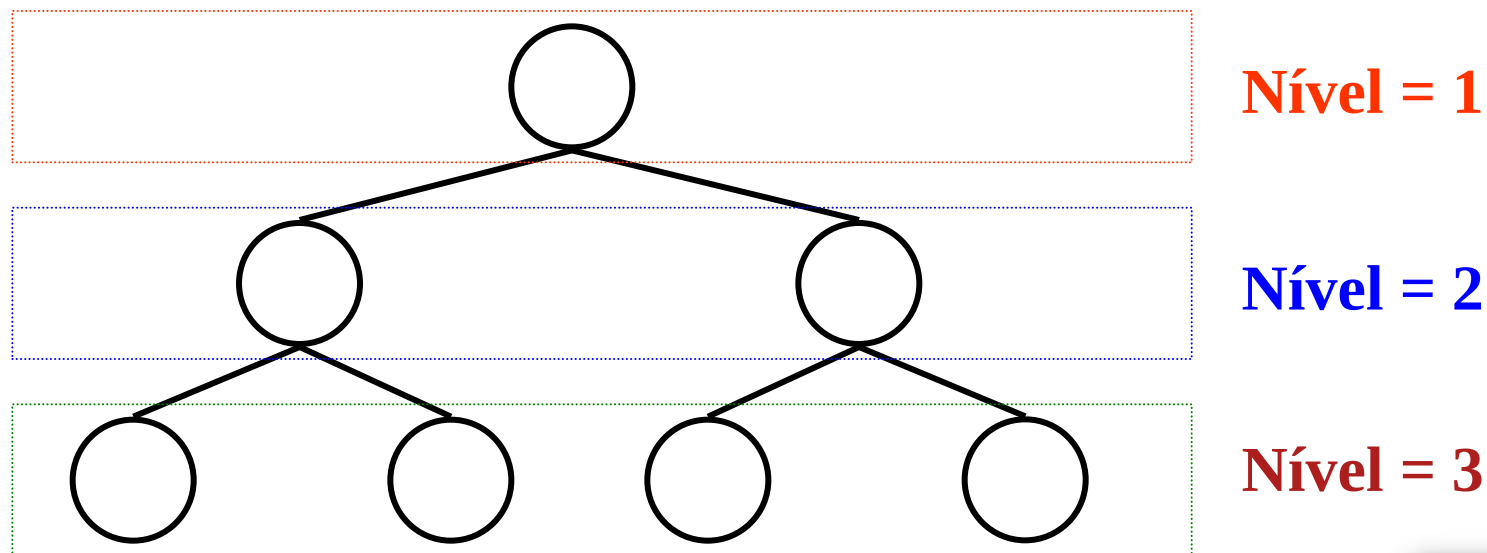


DEFINIÇÃO: NÍVEL

- O **nível** (``level´´) de um nó é o comprimento do caminho da raiz até ao nó + 1, que é o número de nós no caminho.

DEFINIÇÃO: NÍVEL

- Uma árvore vazia tem nível 0.
- A raiz possui o nível 1.





DEFINIÇÃO: ALTURA

- A **altura** (``height``) de uma árvore não vazia é o nível máximo de um nó na árvore.
- Uma árvore vazia tem altura 0.
- Uma árvore só com a raiz possui altura de 1 (o nó é raiz e folha).

DEFINIÇÃO: ALTURA



Escola Superior de Tecnologia e Gestão
Instituto Politécnico da Guarda

Altura = 1

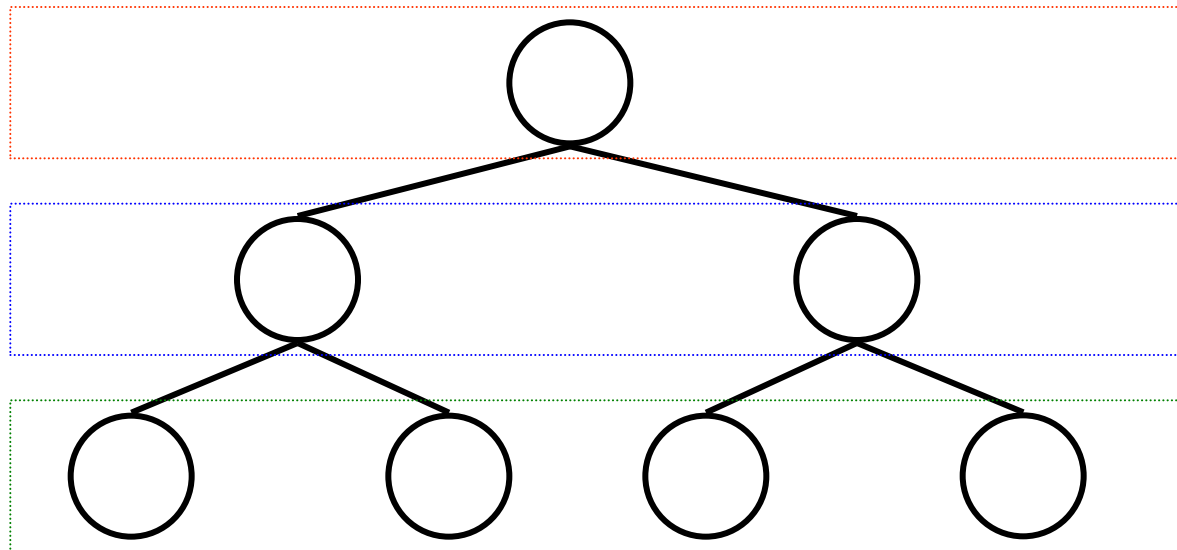
Nível = 1

Altura = 2

Nível = 2

Altura = 3

Nível = 3



Altura da árvore = 3

PROPRIEDADES: N° DE NÓS E FOLHAS

- N° máximo de nós numa árvore = $2^i - 1$
- N° máximo de folhas na árvore = 2^{i-1}
- i – nível

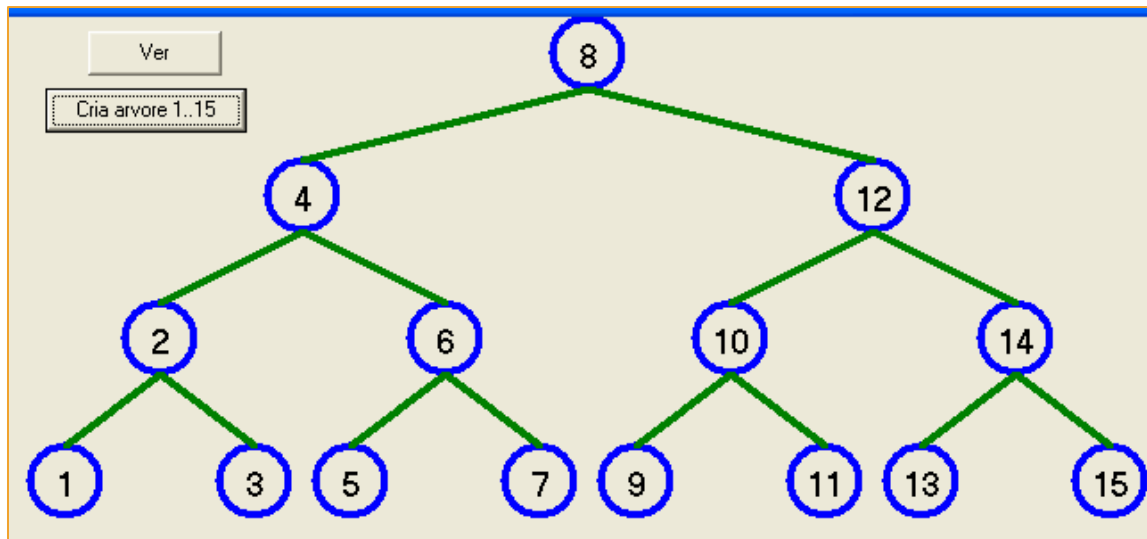
Para $i = 4$, temos:

- N° máximo de nós numa árvore = **15**
- N° máximo de folhas na árvore = **8**



TEOREMAS

- Uma árvore binária com N nós não-terminais possui $N+1$ folhas.
- Uma árvore binária com N nós não-terminais possui $2*N$ arestas ($N-1$ para os nós não-terminais e $N+1$ para as folhas).
- Numa árvore binária com N nós, o nível das folhas (h) varia entre $\log_2 N$ e $N-1$.



Nível 1

Nível 2

Nível 3

Nível 4

Nós não terminais $NNT = 2^{i-1} - 1 = 7$

Nº nós = $2^i - 1 = 15$

Nº folhas = $2^{i-1} = 8$

i – nível = 4

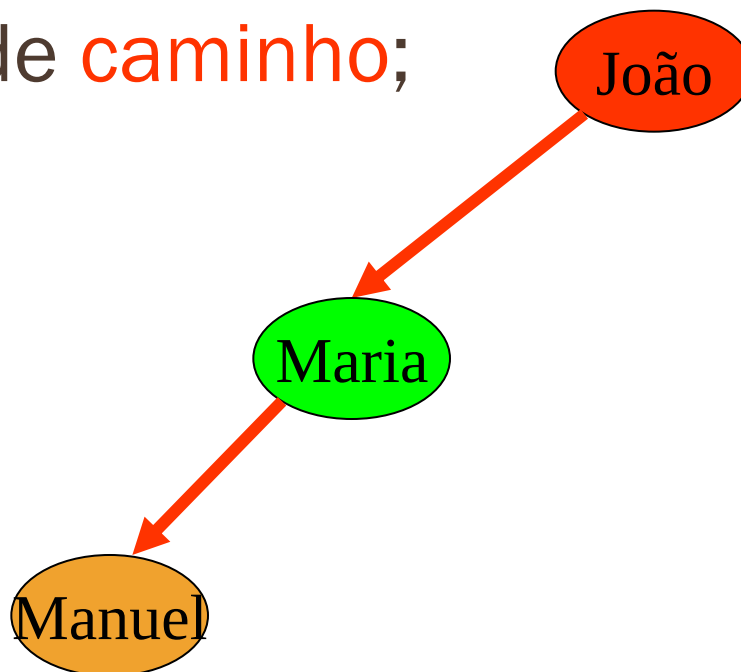
Altura = 4 (máx nível)

Folhas = 8 ($7_{\text{(nós-não terminais)}} + 1$)

Arestas = 14 ($2 * 7$ (NNT))

CAMINHO

- ❑ Cada nó é atingível a partir da raiz através de uma sequência única de arcos, chamados de **caminho**;





NÚMERO DE NÓS: NO NÍVEL

	Nº Nós			
Nível	K=2	K=4	K=8	K=10
1	1	1	1	1
2	2	4	8	10
3	4	16	64	100
4	8	64	512	1.000
5	16	256	4.096	10.000
6	32	1.024	32.768	100.000
7	64	4.096	262.144	1.000.000
8	128	16.384	2.097.152	10.000.000
9	256	65.536	16.777.216	100.000.000
10	512	262.144	134.217.728	1.000.000.000
11	1.024	1.048.576	1.073.741.824	10.000.000.000
12	2.048	4.194.304	8.589.934.592	100.000.000.000
13	4.096	16.777.216	68.719.476.736	1.000.000.000.000
14	8.192	67.108.864	549.755.813.888	10.000.000.000.000
15	16.384	268.435.456	4.398.046.511.104	100.000.000.000.000
16	32.768	1.073.741.824	35.184.372.088.832	1.000.000.000.000.000



NÚMERO DE NÓS: NO NÍVEL

	Nº Nós		
Nível	K=2	K=4	K=8
17	65.536	4.294.967.296	281.474.976.710.656
18	131.072	17.179.869.184	2.251.799.813.685.250
19	262.144	68.719.476.736	18.014.398.509.482.000
20	524.288	274.877.906.944	144.115.188.075.856.000
21	1.048.576	1.099.511.627.776	1.152.921.504.606.850.000
22	2.097.152	4.398.046.511.104	9.223.372.036.854.780.000
23	4.194.304	17.592.186.044.416	73.786.976.294.838.200.000
24	8.388.608	70.368.744.177.664	590.295.810.358.706.000.000
25	16.777.216	281.474.976.710.656	4.722.366.482.869.650.000.000
26	33.554.432	1.125.899.906.842.620	37.778.931.862.957.200.000.000
27	67.108.864	4.503.599.627.370.500	302.231.454.903.657.000.000.000
28	134.217.728	18.014.398.509.482.000	2.417.851.639.229.260.000.000.000
29	268.435.456	72.057.594.037.927.900	19.342.813.113.834.100.000.000.000
30	536.870.912	288.230.376.151.712.000	154.742.504.910.673.000.000.000.000
31	1.073.741.824	1.152.921.504.606.850.000	1.237.940.039.285.380.000.000.000.000
32	2.147.483.648	4.611.686.018.427.390.000	9.903.520.314.283.040.000.000.000.000



PESQUISA NUMA ÁRVORE BINÁRIA

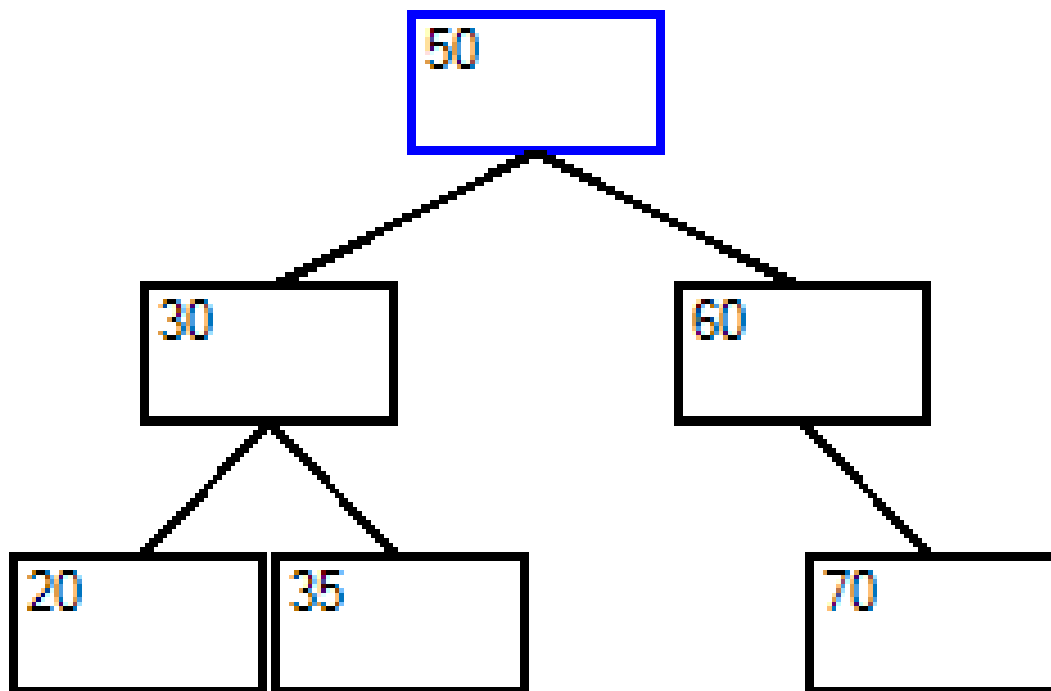
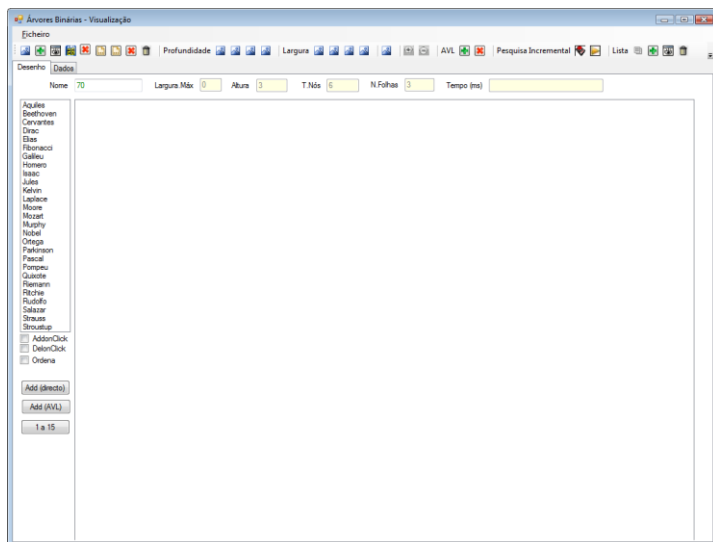
- ❑ Para cada nó, comparar a chave a ser localizada com o valor armazenado no nó corrente apontado.
- ❑ Se a chave for menor que o valor, vá para a subárvore à esquerda e tente novamente.
- ❑ Se a chave for maior que o valor, vá para a subárvore à direita e tente novamente.
- ❑ Se for a mesma a pesquisa termina.



PERCURSO EM ÁRVORES

- ❑ O percurso em árvores é o processo de visitar cada nó da árvore exactamente uma vez.
- ❑ O percurso pode ser interpretado como colocar todos os nós numa linha.

EXEMPLO



50 30 60 20 35 70



PERCURSO: EXTENSÃO

□ Ordem

□ esquerda,

□ 1º nível, 2º nível, ..., nº nível

□ nº nível, n-1º nível, ..., 1º nível

50 30 60 20 35 70

20 35 70 30 60 50

□ Direita,

□ 1º nível, 2º nível, ..., nº nível

□ nº nível, n-1º nível, ..., 1º nível

50 60 30 70 35 20

70 35 20 60 30 50

PERCURSO: PROFUNDIDADE

V – visitar um nó;

L – percorrer a subárvore esquerda (left)

R – percorrer a subárvore direita (right)

□ Ordem ($3! = 6$, formas de percorrer a árvore)

1. VLR 50 30 20 35 60 70

2. VRL

3. LVR

4. RVL

5. LRV

6. RLV



PERCURSO: PROFUNDIDADE

VLR - percurso em pré-ordem;

LVR - percurso em in-ordem;

LRV - percurso em pós-ordem;

50 30 20 35 60 70



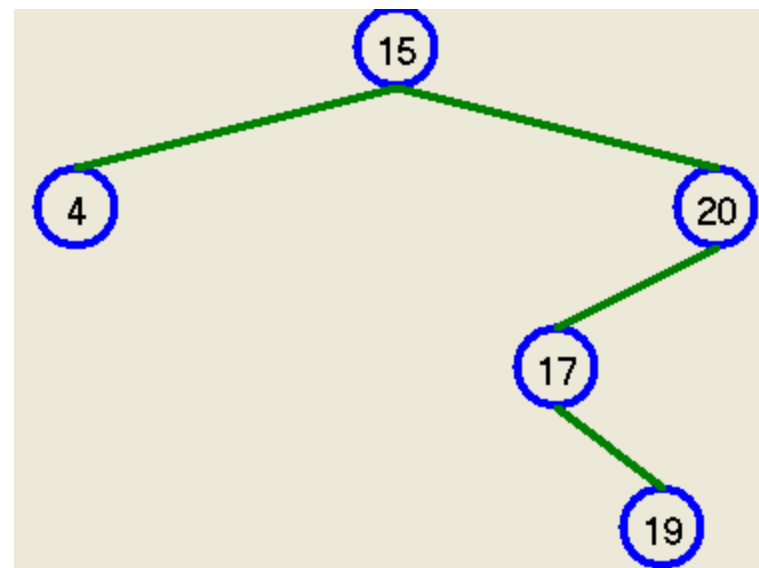
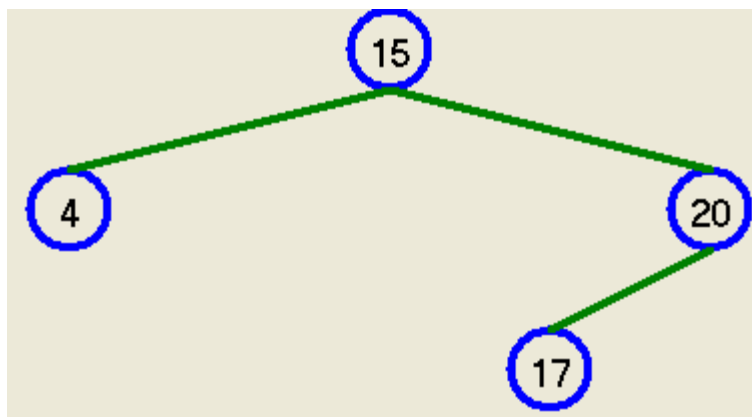
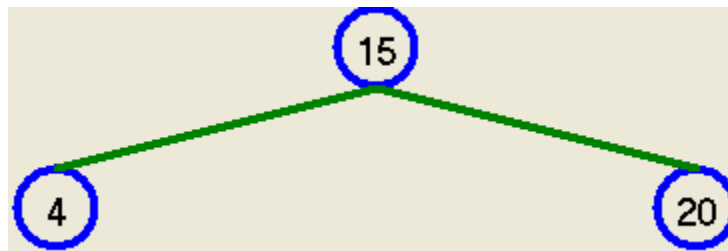
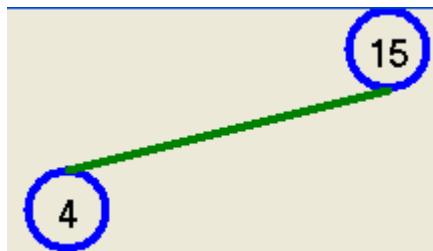
INSERÇÃO

- ❑ Caso 1: a árvore está vazia
 - ❑ Criar um novo nó;
 - ❑ Raiz = novo;
- ❑ Caso 2: A árvore já tem nós:
 - ❑ Efectuar uma pesquisa na árvore com dois ponteiros: pai e filho. No final o ponteiro aponta para uma folha. O novo elemento é inserido à esquerda se for menor que a folha. É inserido à direita se for maior do que a folha.
 - ❑ o novo nó é inserido sempre num folha;

INSERÇÃO



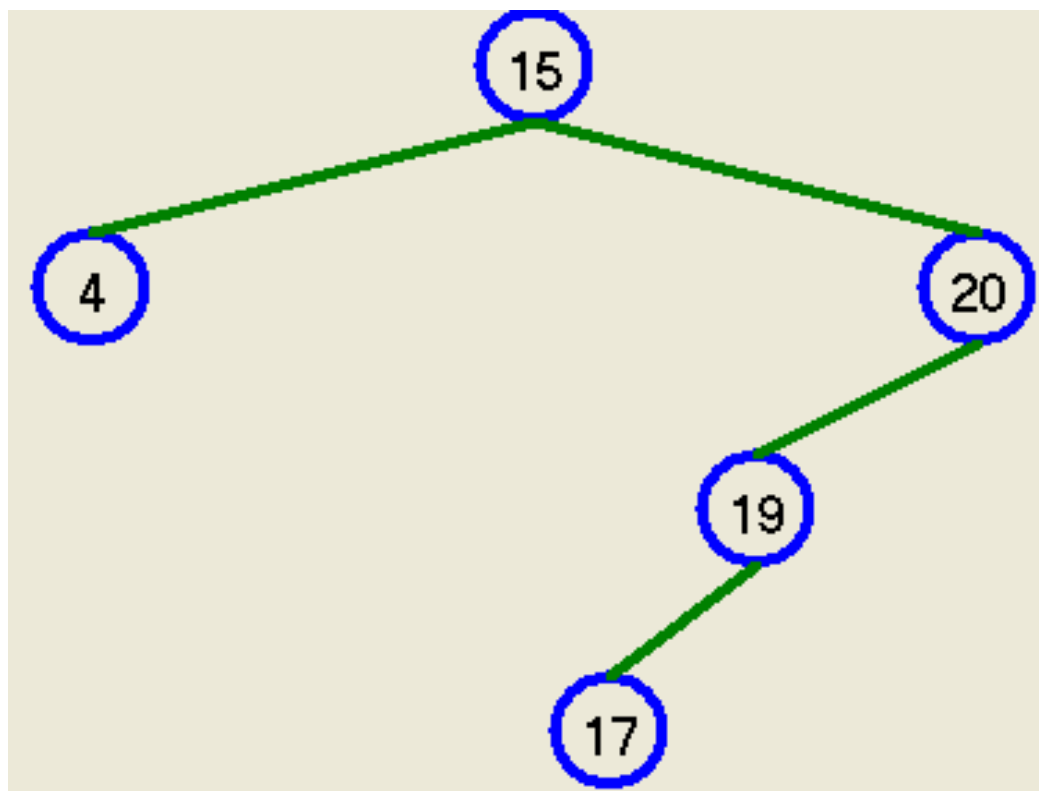
Escola Superior de Tecnologia e Gestão
Instituto Politécnico da Guarda



15 4 20 17 19

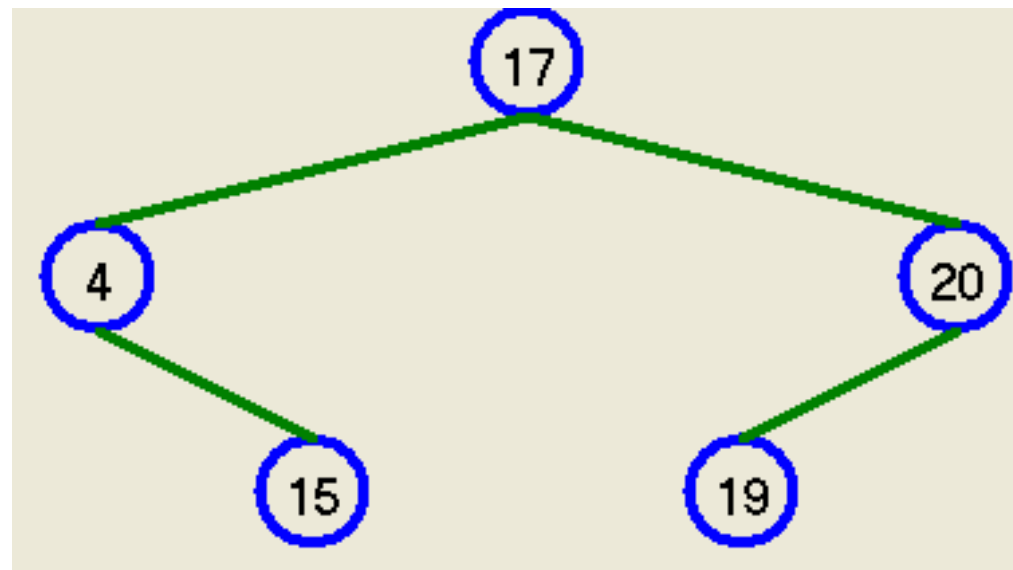
INSERÇÃO

`int v[] = {15,20,4,19,17};`



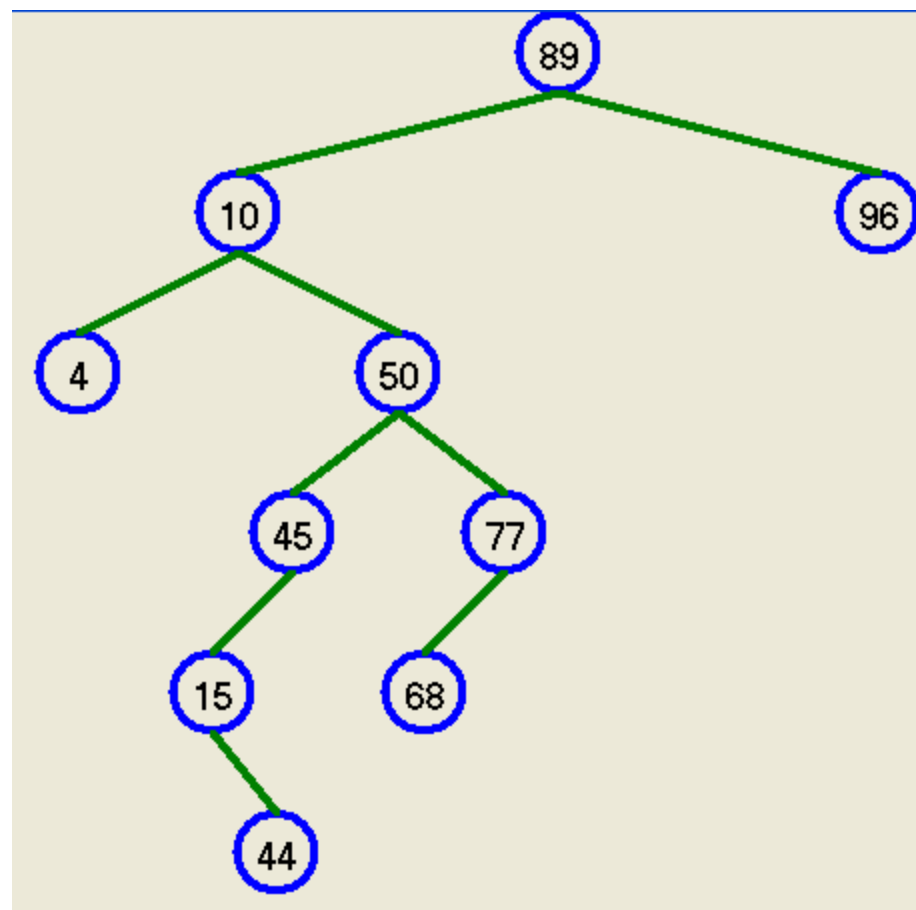
INSERÇÃO

```
int v[] = {17,20,4,19,15};
```



INSERÇÃO: EXERCÍCIO

`int v[] = {89,10,50,45,15,77,44,68,4,96};`

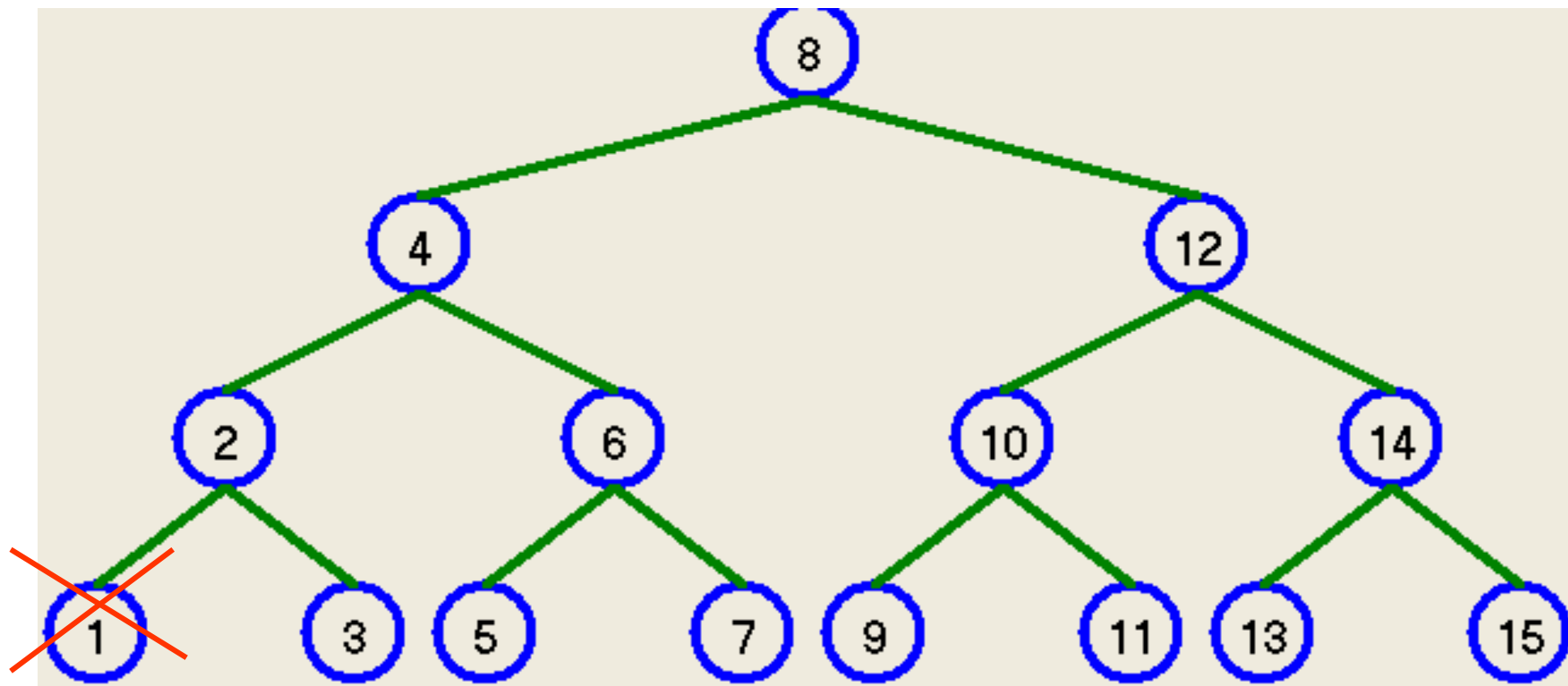




REMOÇÃO

- ❑ O nó a remover pode ser:
 - ❑ uma folha (operação simples);
 - ❑ Um nó não terminal:
 - ❑ O nó tem um filho. Este caso não é complicado.
 - ❑ O nó tem dois filhos. O mais difícil.

NA FOLHA: À ESQUERDA



Ponteiro para 2 (pai)

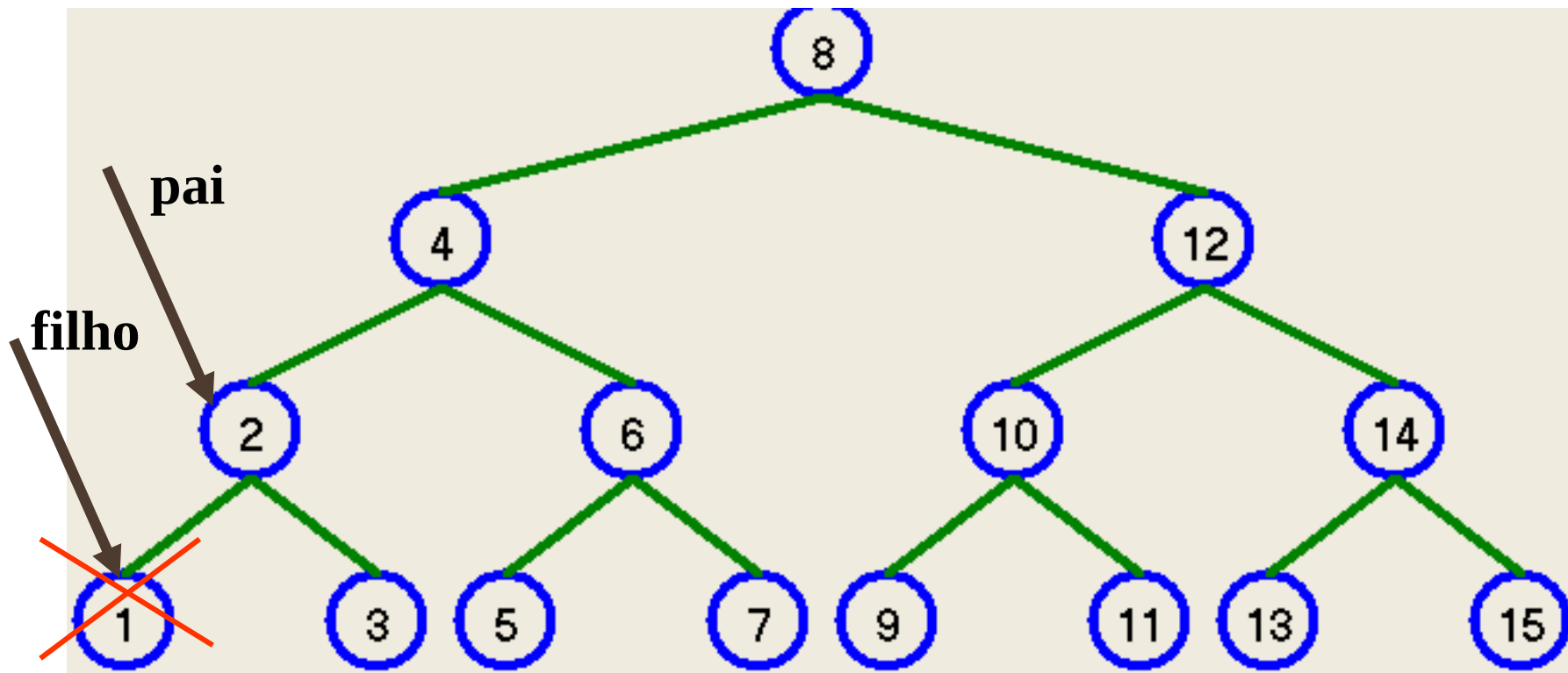
Ponteiro para 1 (filho)

filho->esq = 0 && filho->dir = 0

pai->esq = 0;

Delete filho;

NA FOLHA: À ESQUERDA



Ponteiro para 2 (pai)

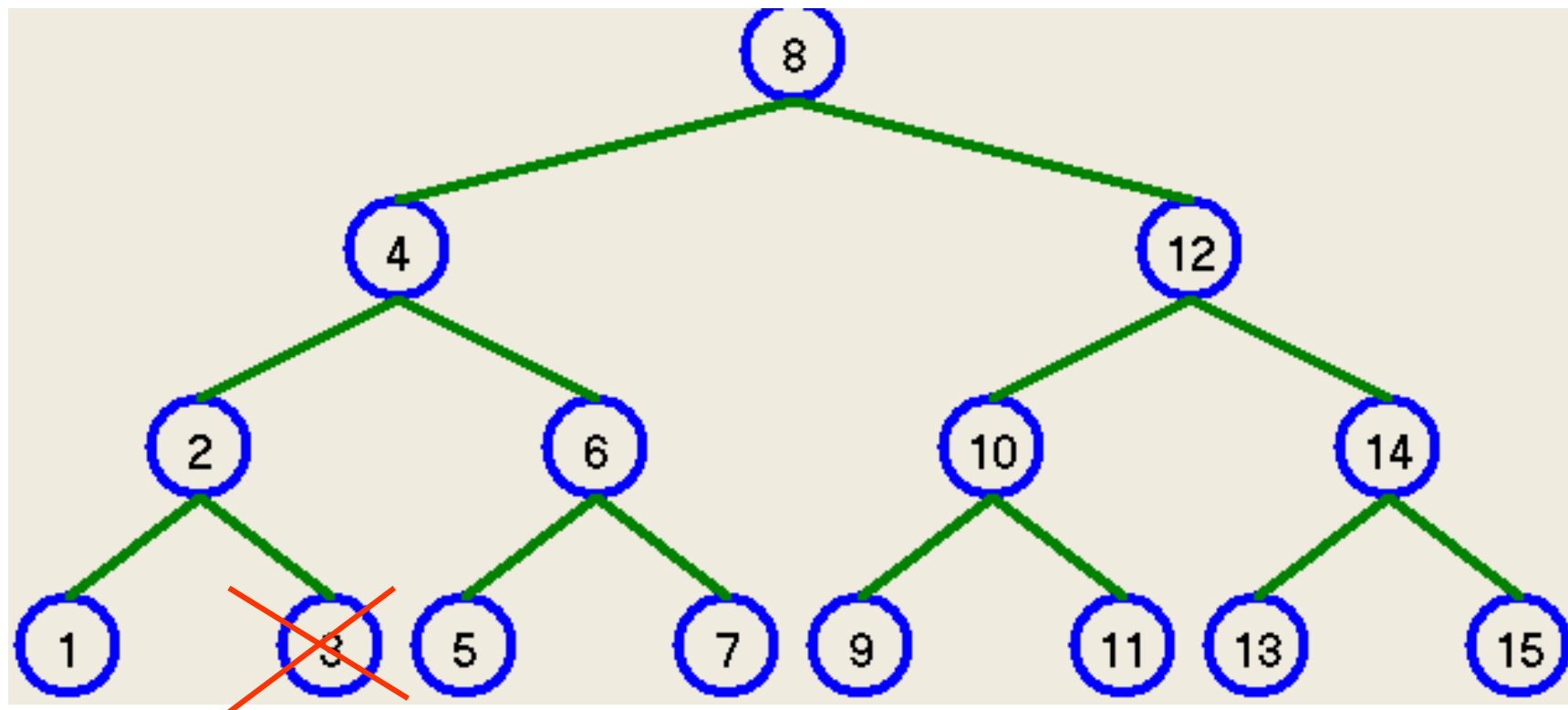
Ponteiro para 1 (filho)

$\text{filho} \rightarrow \text{esq} = 0 \ \&\& \ \text{filho} \rightarrow \text{dir} = 0$

$\text{pai} \rightarrow \text{esq} = 0;$

Delete filho;

NA FOLHA: À DIREITA



Ponteiro para 2 (pai)

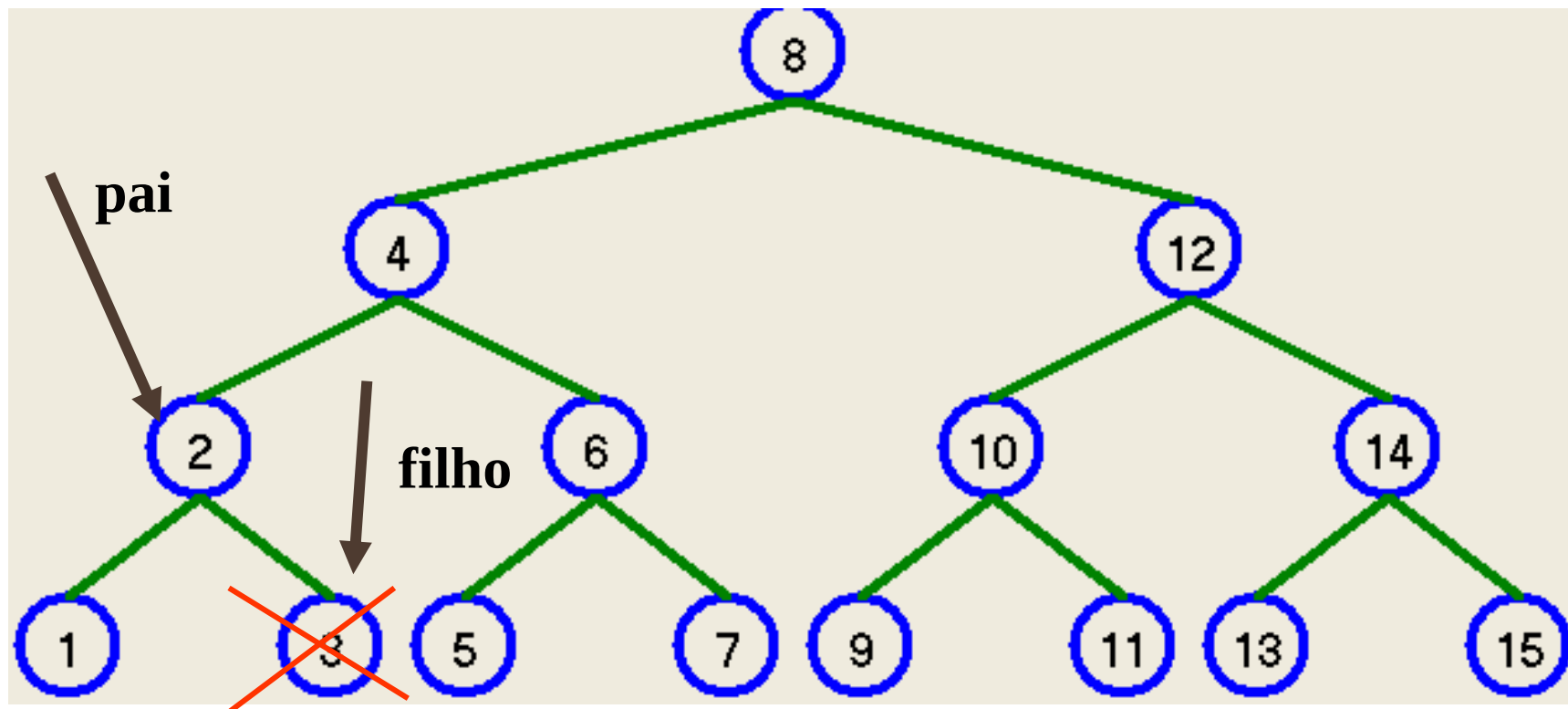
Ponteiro para 1 (filho)

filho->esq = 0 && filho->dir = 0

pai->dir = 0;

Delete filho;

NA FOLHA: À DIREITA



Ponteiro para 2 (pai)

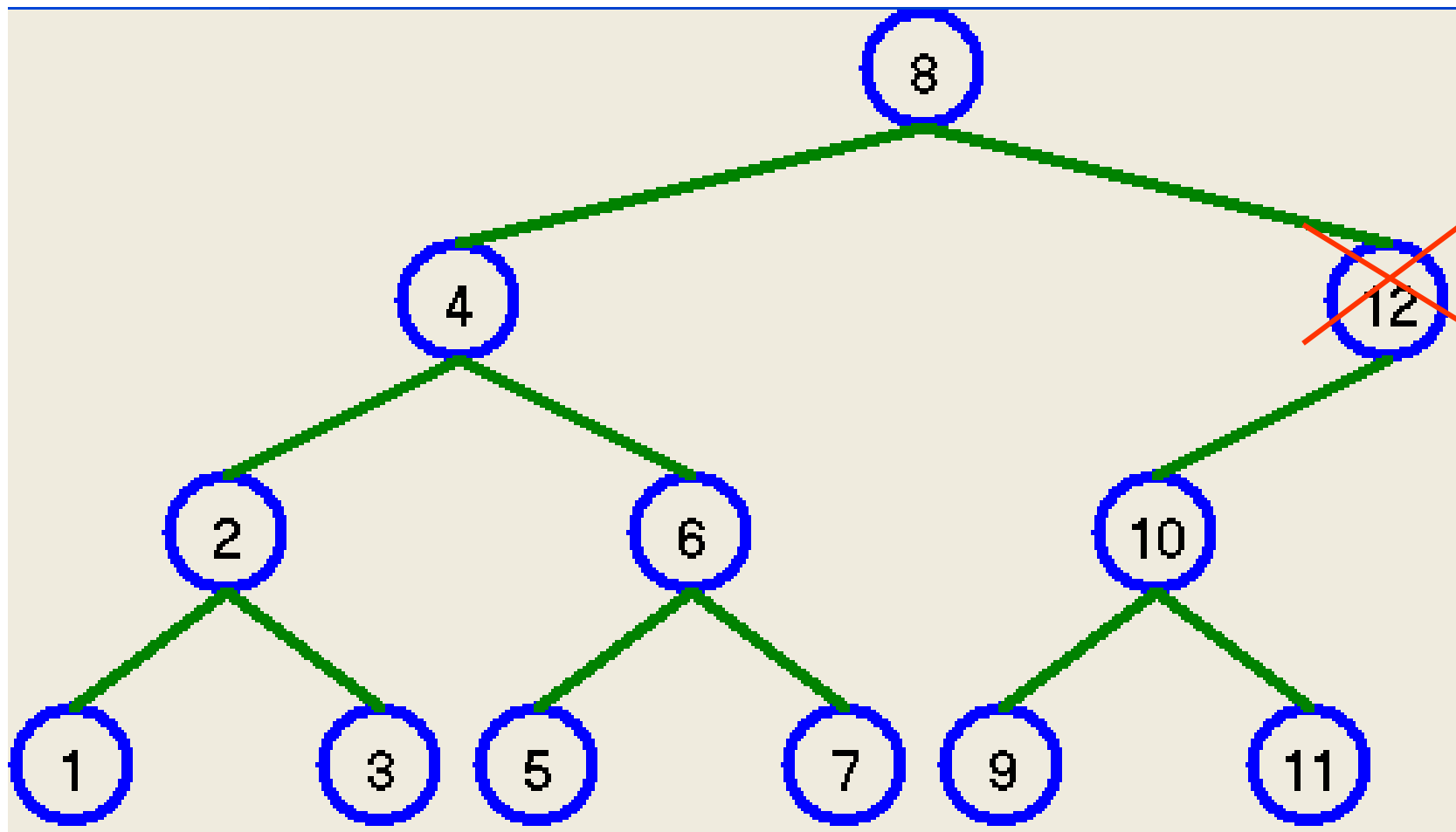
Ponteiro para 1 (filho)

filho->esq = 0 && filho->dir = 0

pai->dir = 0;

Delete filho;

NO NÃO TERMINAL: FILHO ESQ

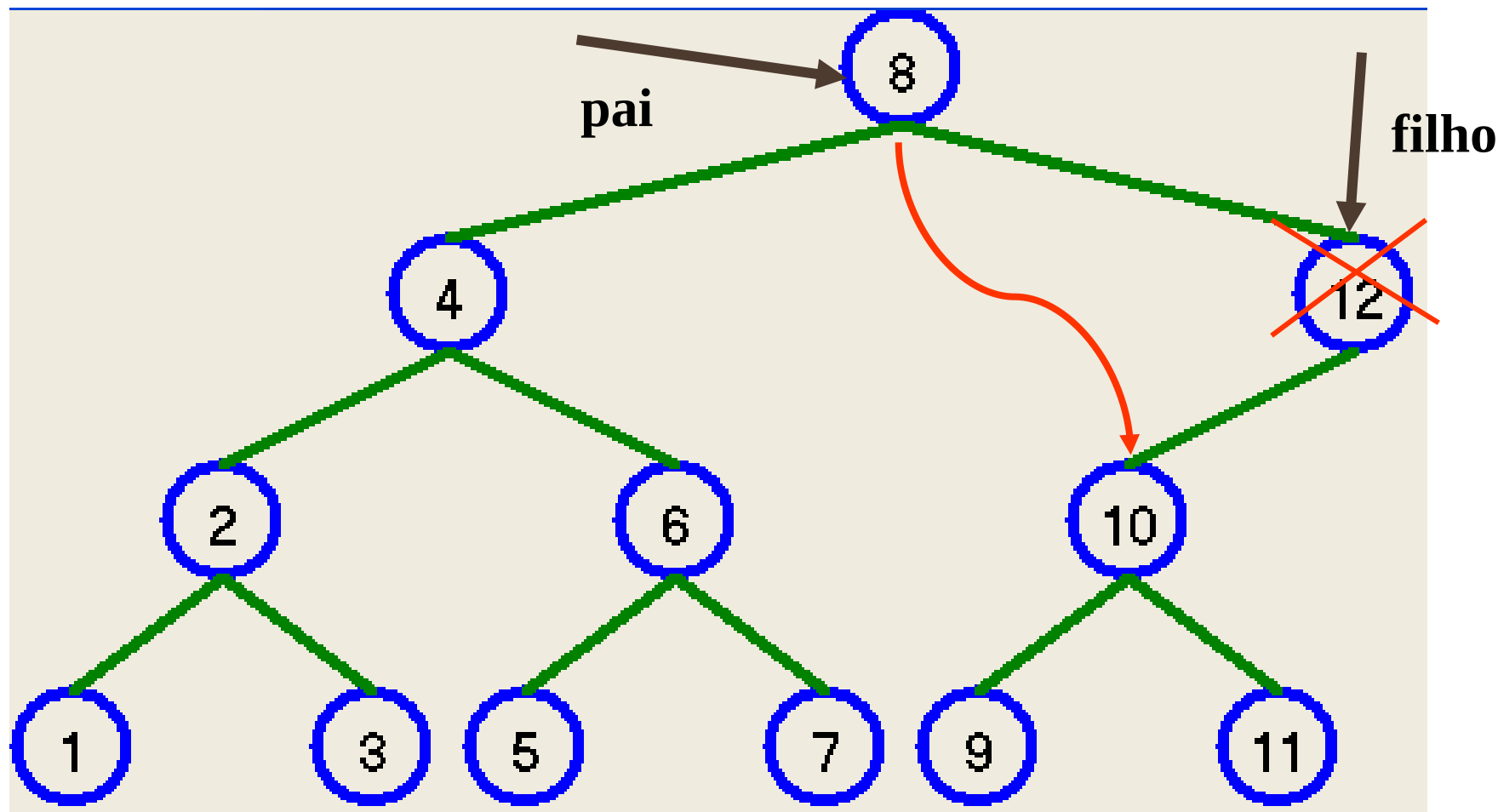




NO NÃO TERMINAL: FILHO ESQ (12)

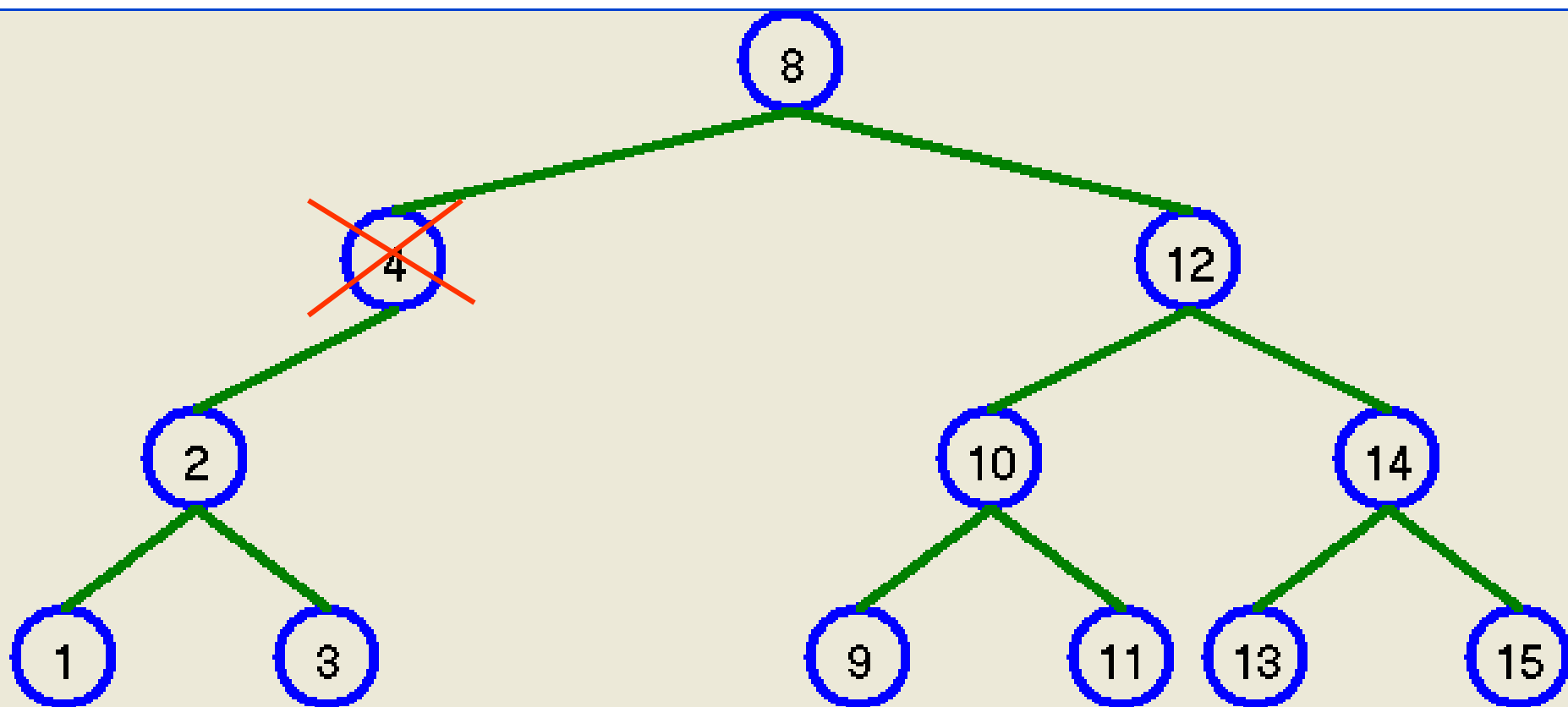
- ❑ O nó a eliminar está á direita do pai.
- ❑ Ligar o nó pai (dir) ao nó filho (esq).
- ❑ Remover o nó filho.

NO NÃO TERMINAL: FILHO ESQ



pai->dir = filho->esq
delete filho

NO NÃO TERMINAL: FILHO ESQ (12)

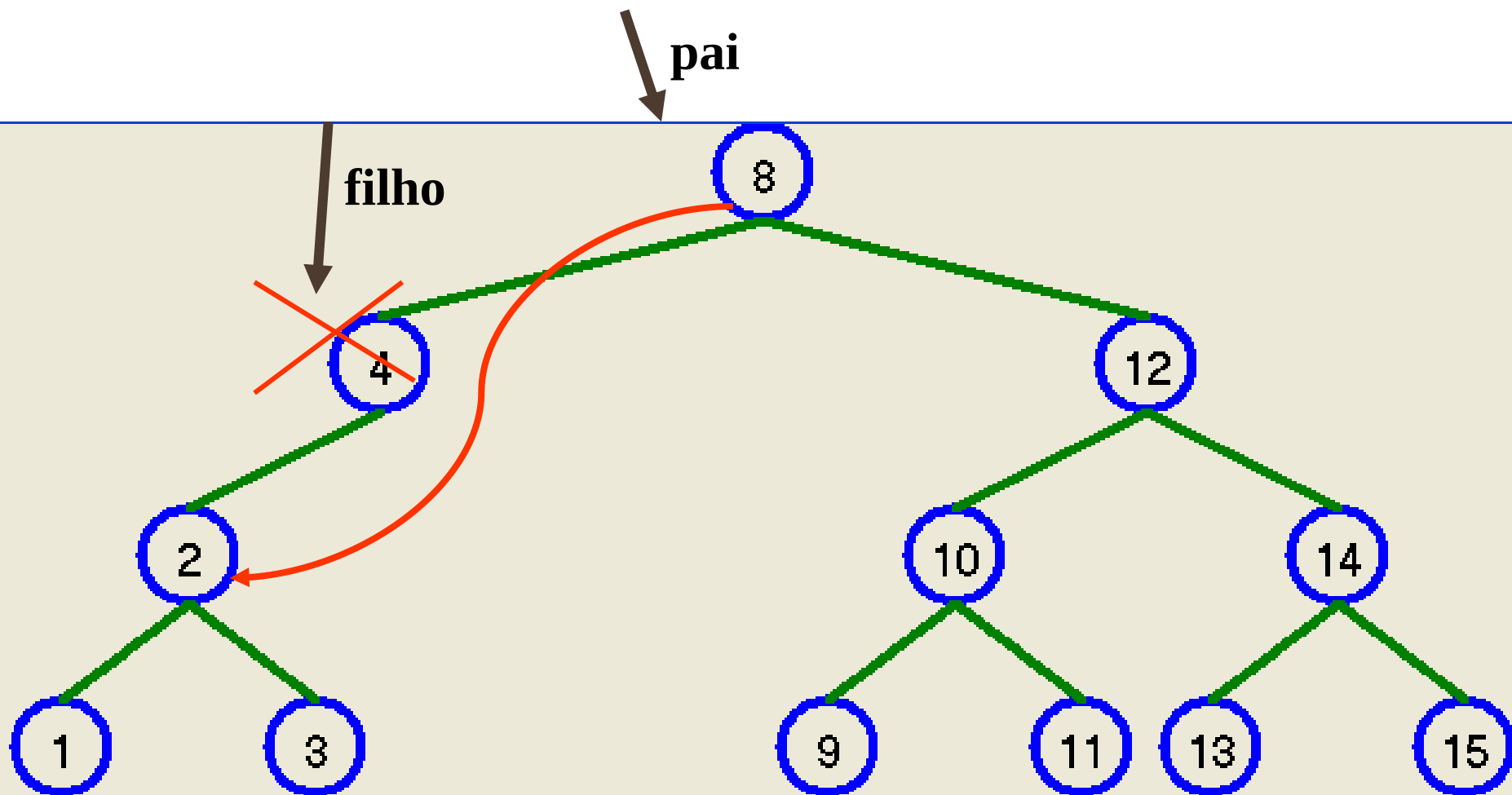




NO NÃO TERMINAL: FILHO ESQ (12)

- ❑ O nó a eliminar está á esquerda do pai.
- ❑ Ligar o nó pai (esq) ao nó filho (esq).
- ❑ Remover o nó filho.

NO NÃO TERMINAL: FILHO ESQ (12)

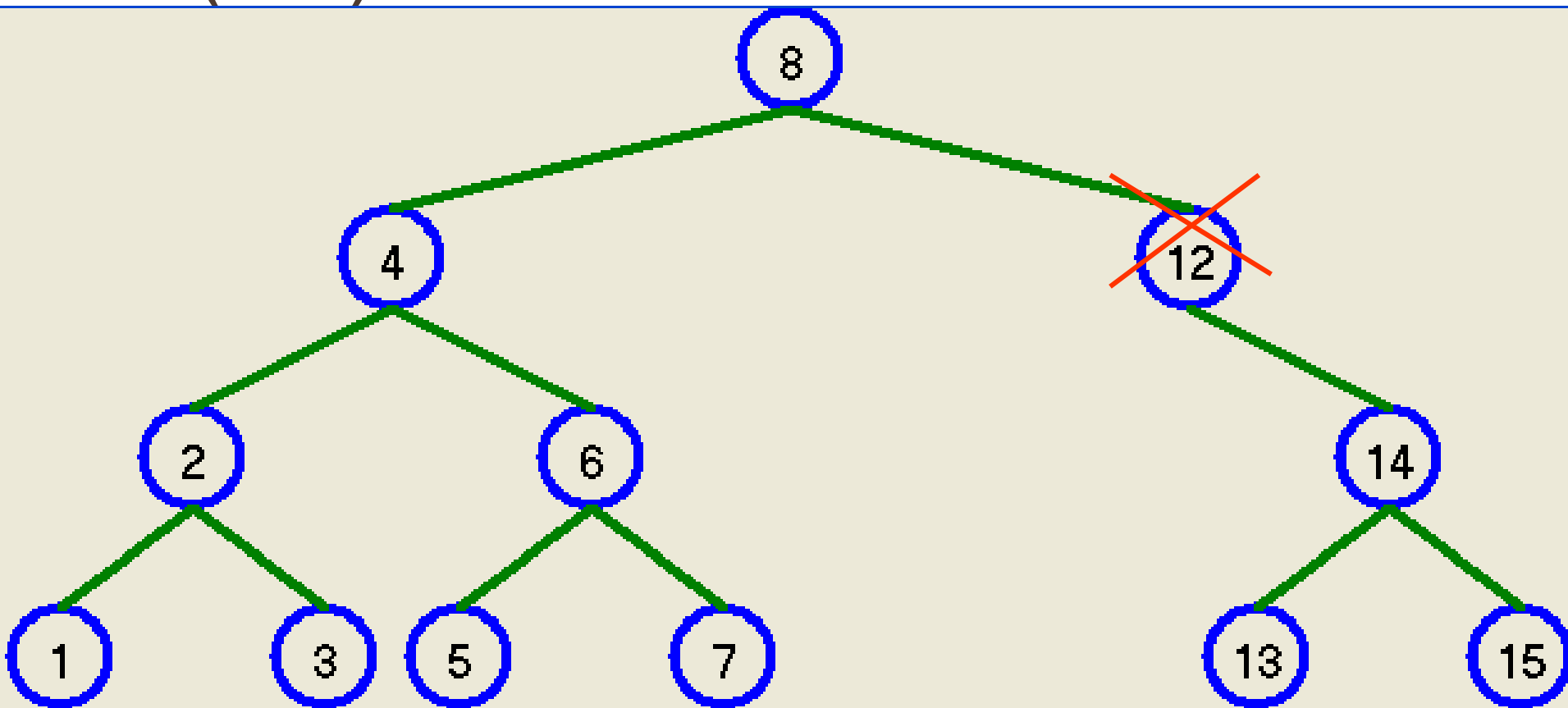


**pai->esq = filho->esq
delete filho**

NO NÃO TERMINAL: FILHO DIR, PAI (DIR)



Escola Superior de Tecnologia e Gestão
Instituto Politécnico da Guarda

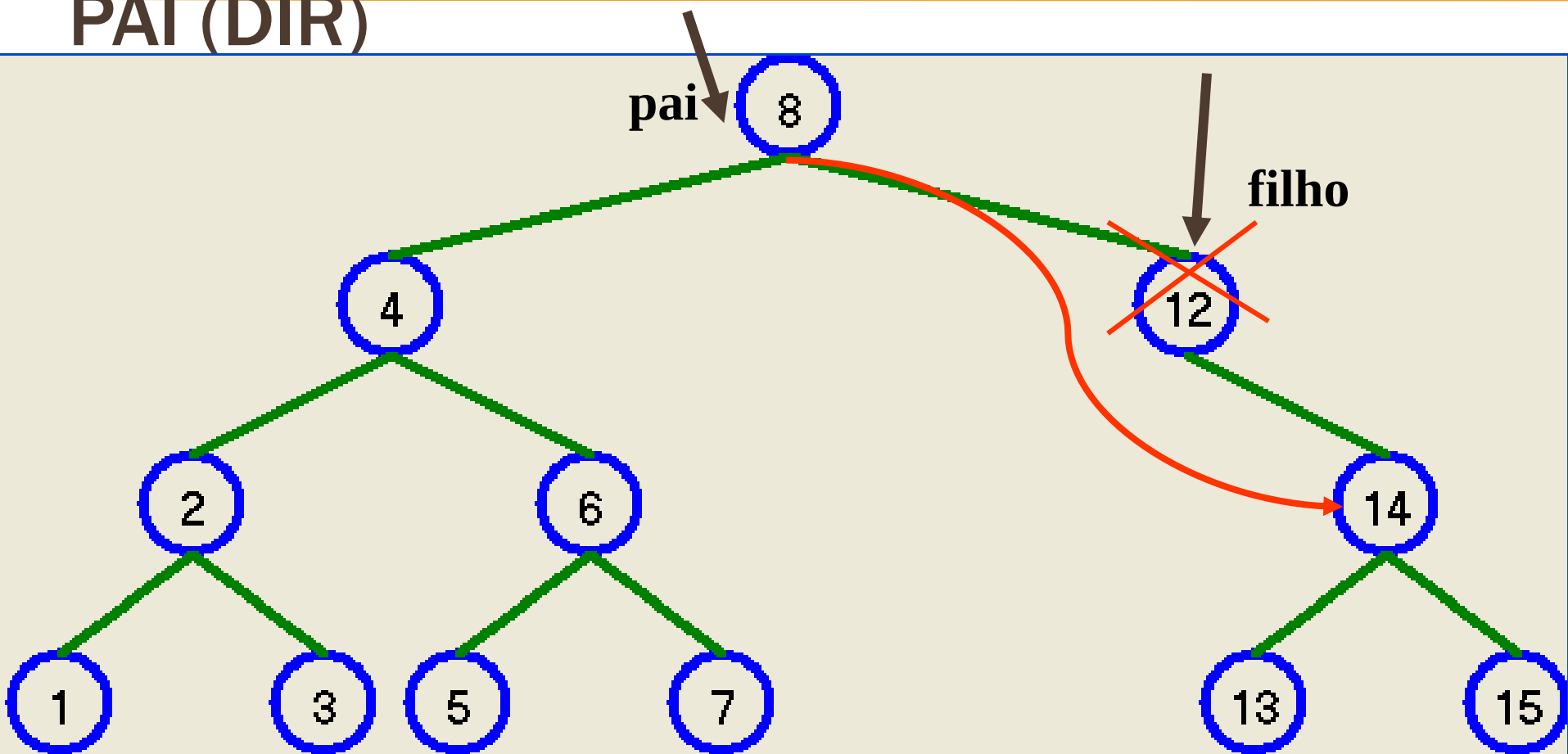




NO NÃO TERMINAL: FILHO DIR, PAI (DIR)

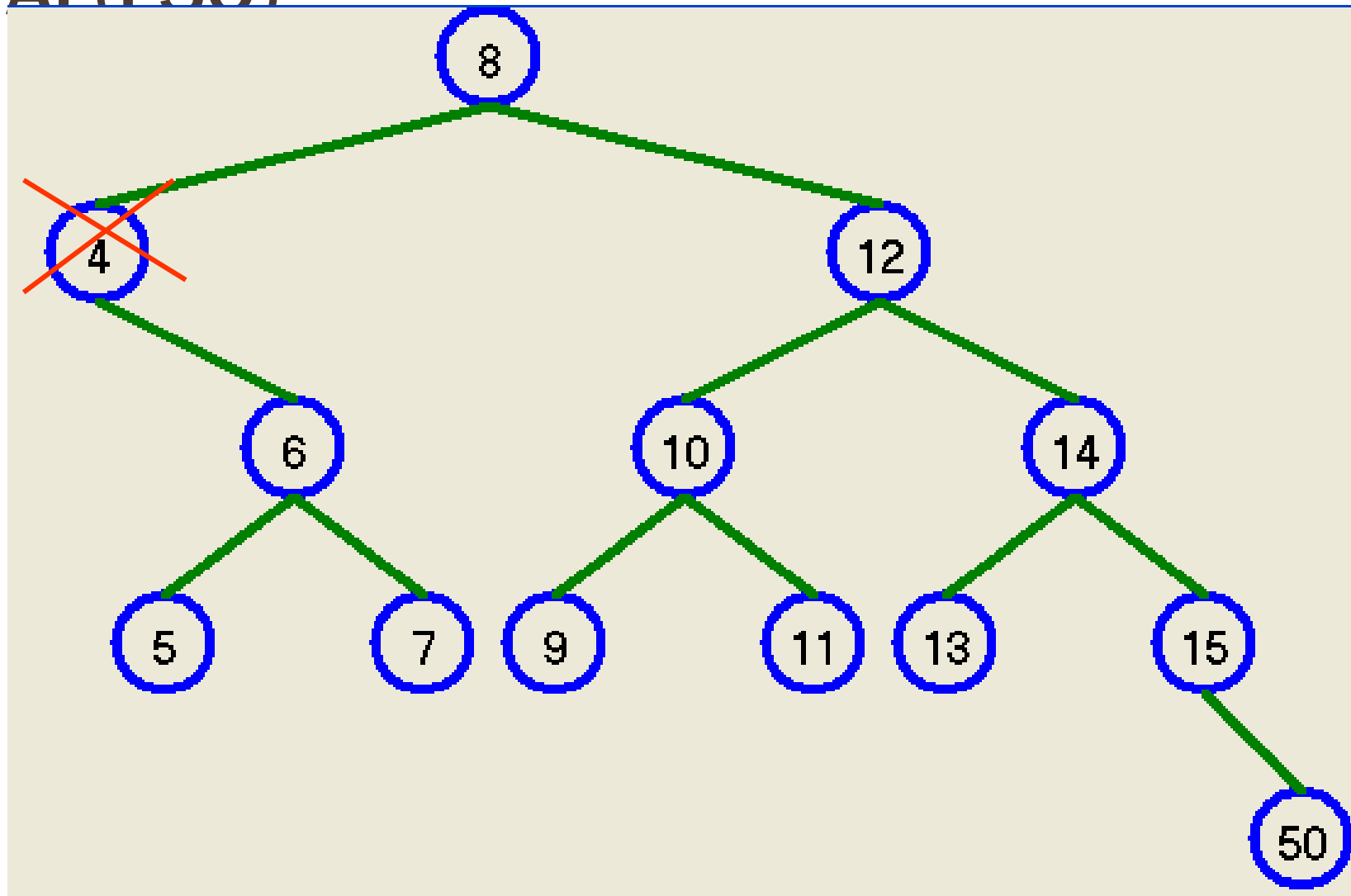
- ❑ O nó a eliminar está á direita do pai.
- ❑ Ligar o nó pai (dir) ao nó filho (dir).
- ❑ Remover o nó filho.

NO NÃO TERMINAL: FILHO DIR, PAI (DIR)



**pai->dir = filho->dir
delete filho**

NO NÃO TERMINAL: FILHO DIR, PAI (FSO)





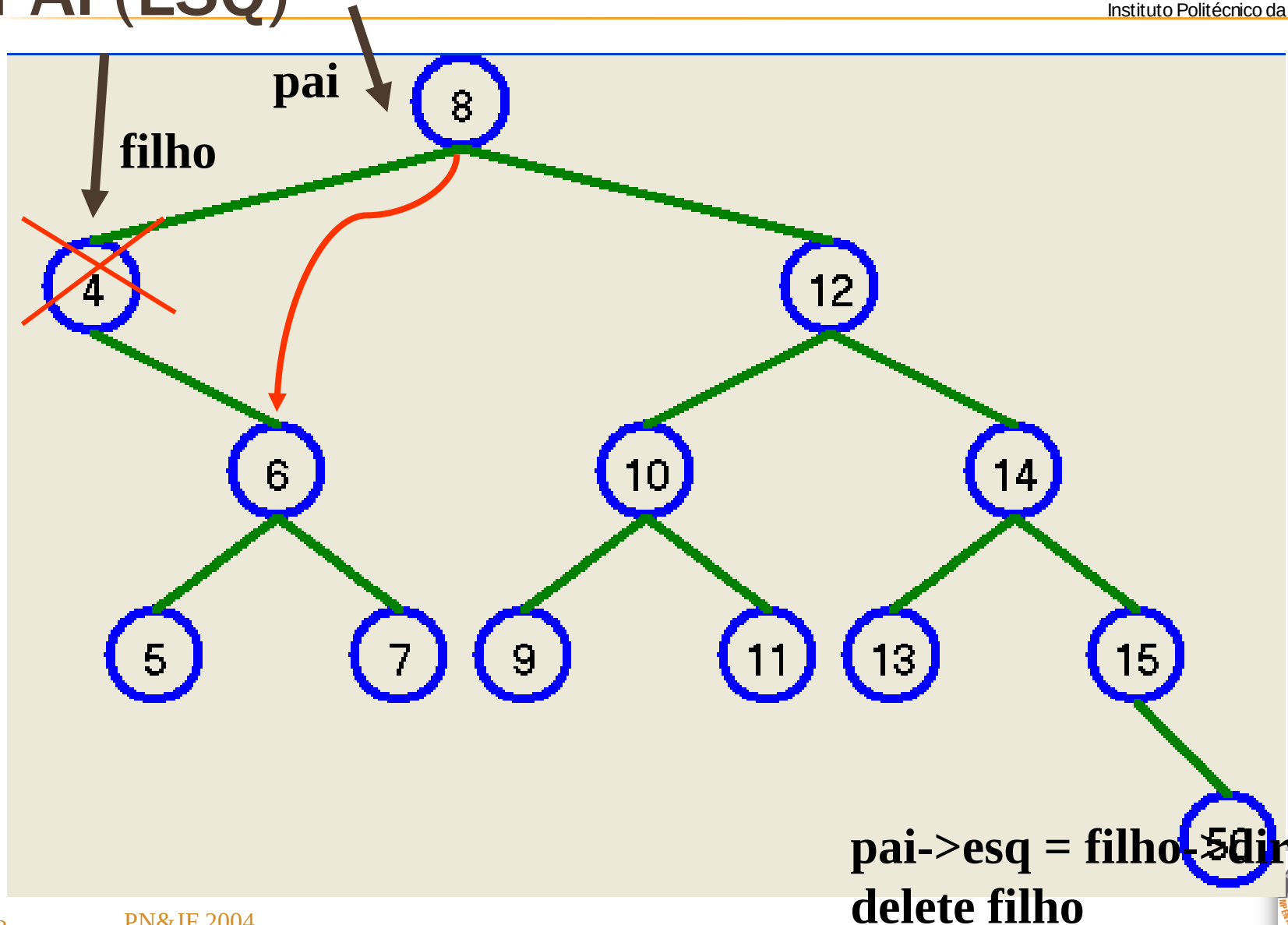
NO NÃO TERMINAL: FILHO DIR, PAI (ESQ)

- ❑ O nó a eliminar está á esquerda do pai.
- ❑ Ligar o nó pai (esq) ao nó filho (dir).
- ❑ Remover o nó filho.

NO NÃO TERMINAL: FILHO DIR, PAI (ESQ)



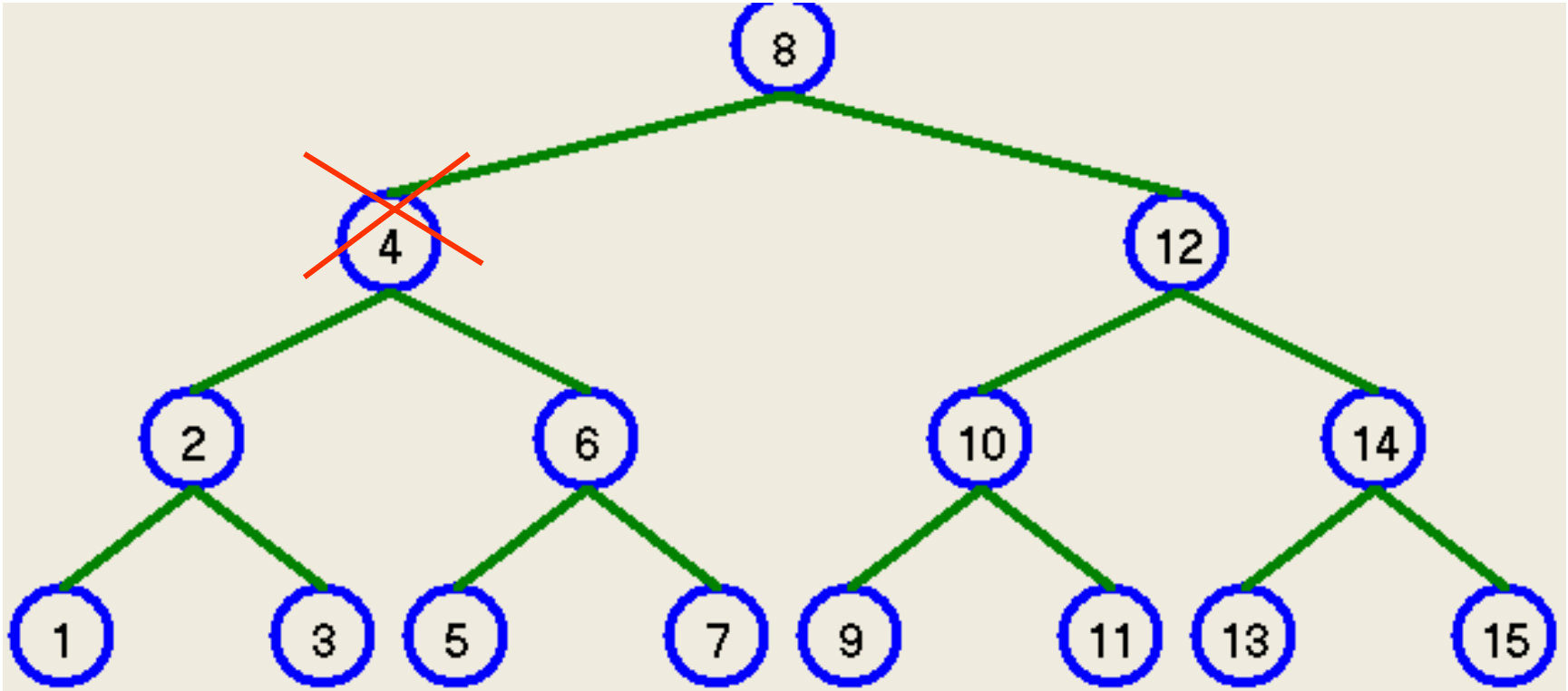
Escola Superior de Tecnologia e Gestão
Instituto Politécnico da Guarda



NO NÃO TERMINAL: FILHO ESQ E DIR, PAI (ESQ)



Escola Superior de Tecnologia e Gestão
Instituto Politécnico da Guarda





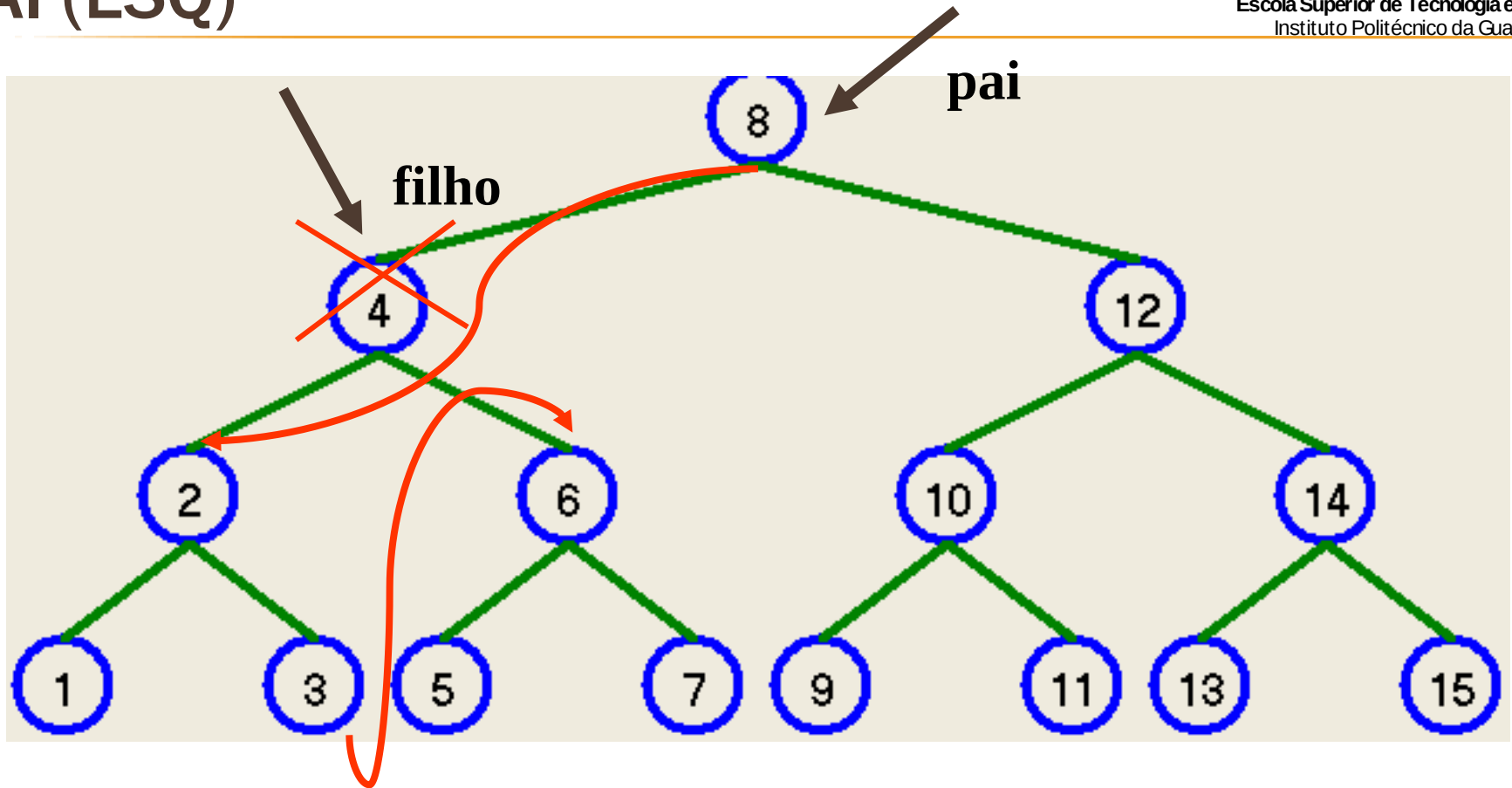
NO NÃO TERMINAL: FILHO ESQ E DIR, PAI (ESQ)

- ❑ O nó filho está à esquerda do pai.
- ❑ Ligar pai (esq) com filho (esq)
- ❑ Ligar o elemento mais à direita da esquerda do filho com o filho (dir).

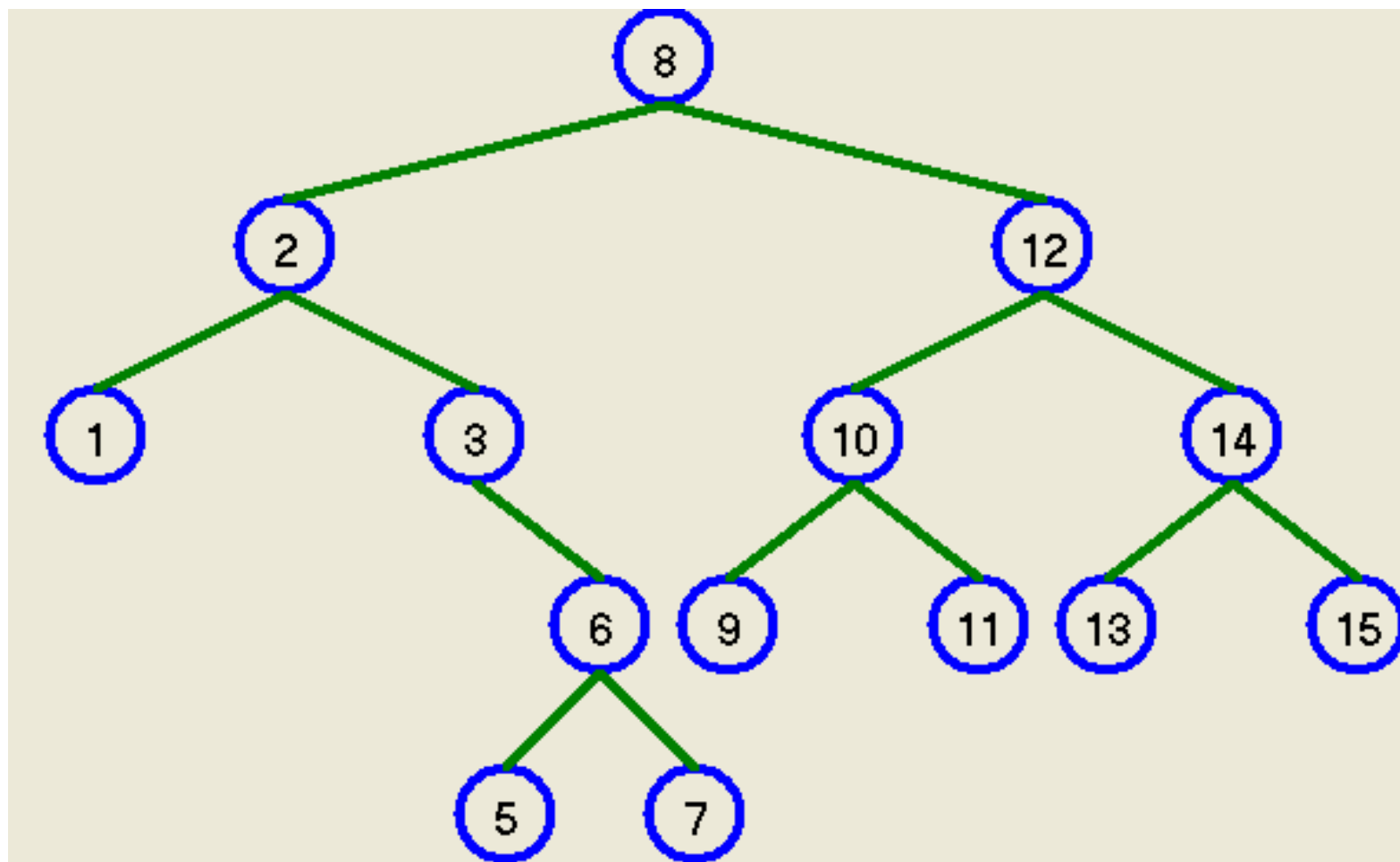
NO NÃO TERMINAL: FILHO ESQ E DIR, PAI (ESQ)



Escola Superior de Tecnologia e Gestão
Instituto Politécnico da Guarda



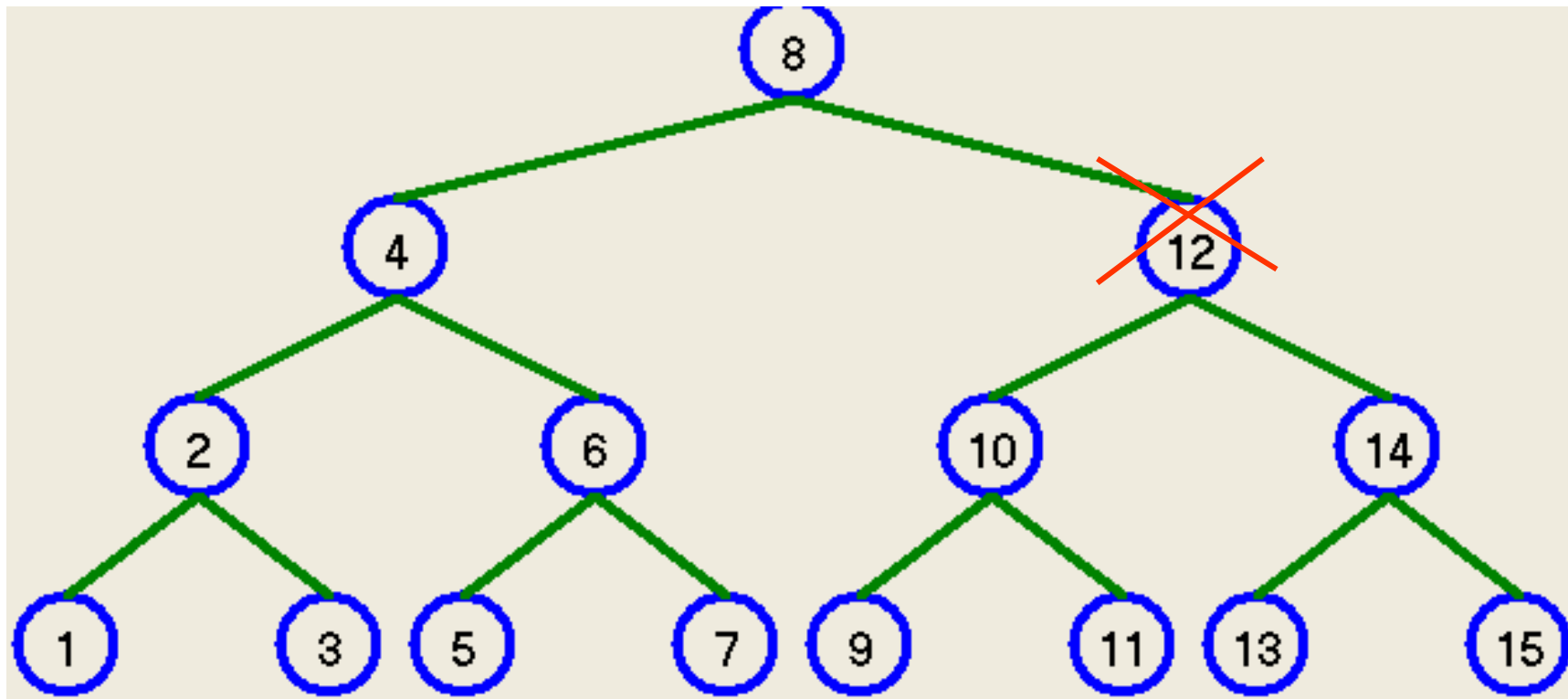
NO NÃO TERMINAL: FILHO ESQ E DIR, PAI (ESQ)



NO NÃO TERMINAL: FILHO ESQ E DIR, PAI (DIR)



Escola Superior de Tecnologia e Gestão
Instituto Politécnico da Guarda



NO NÃO TERMINAL: FILHO ESQ E DIR, PAI (ESQ)

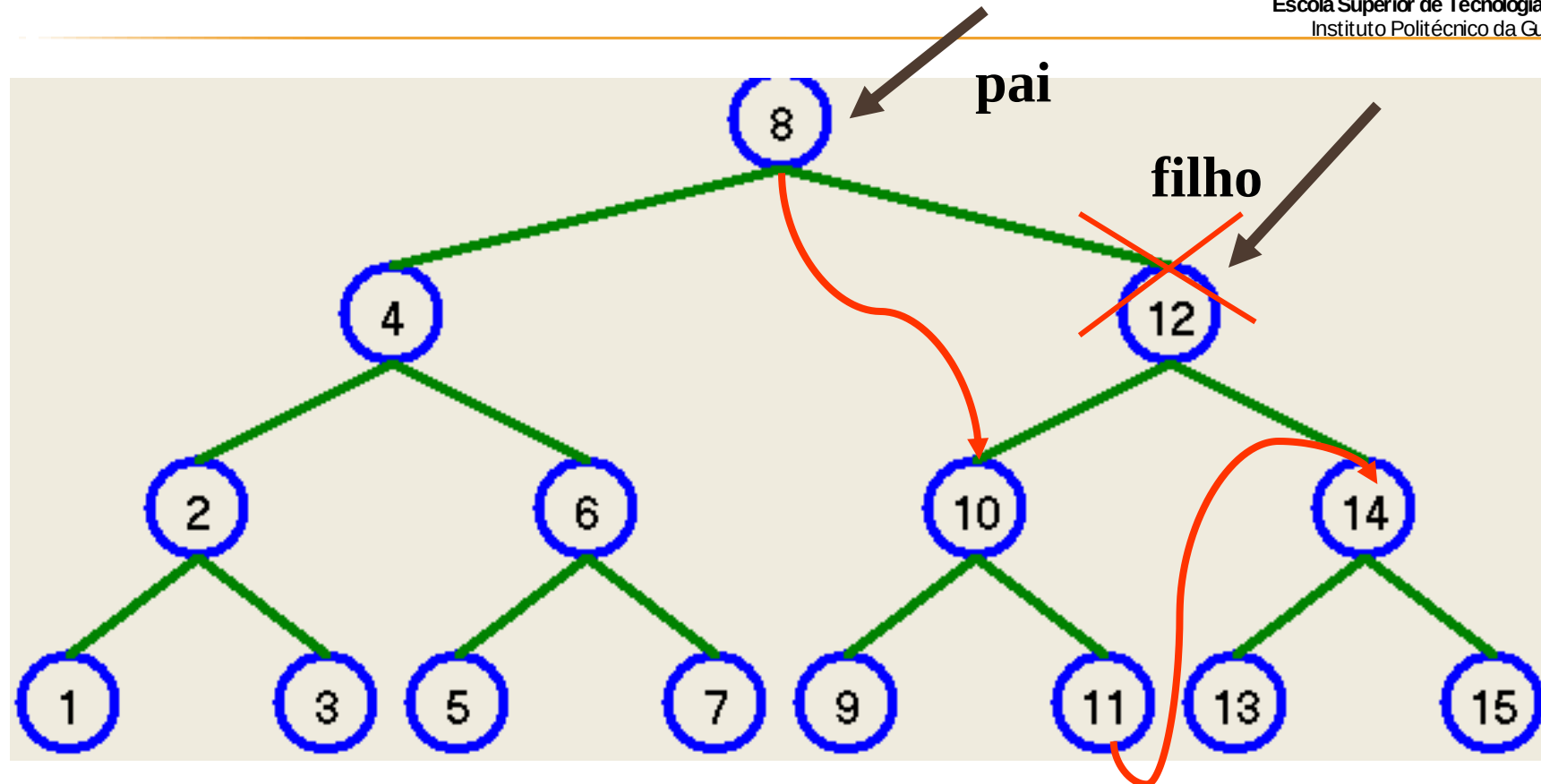


Escola Superior de Tecnologia e Gestão
Instituto Politécnico da Guarda

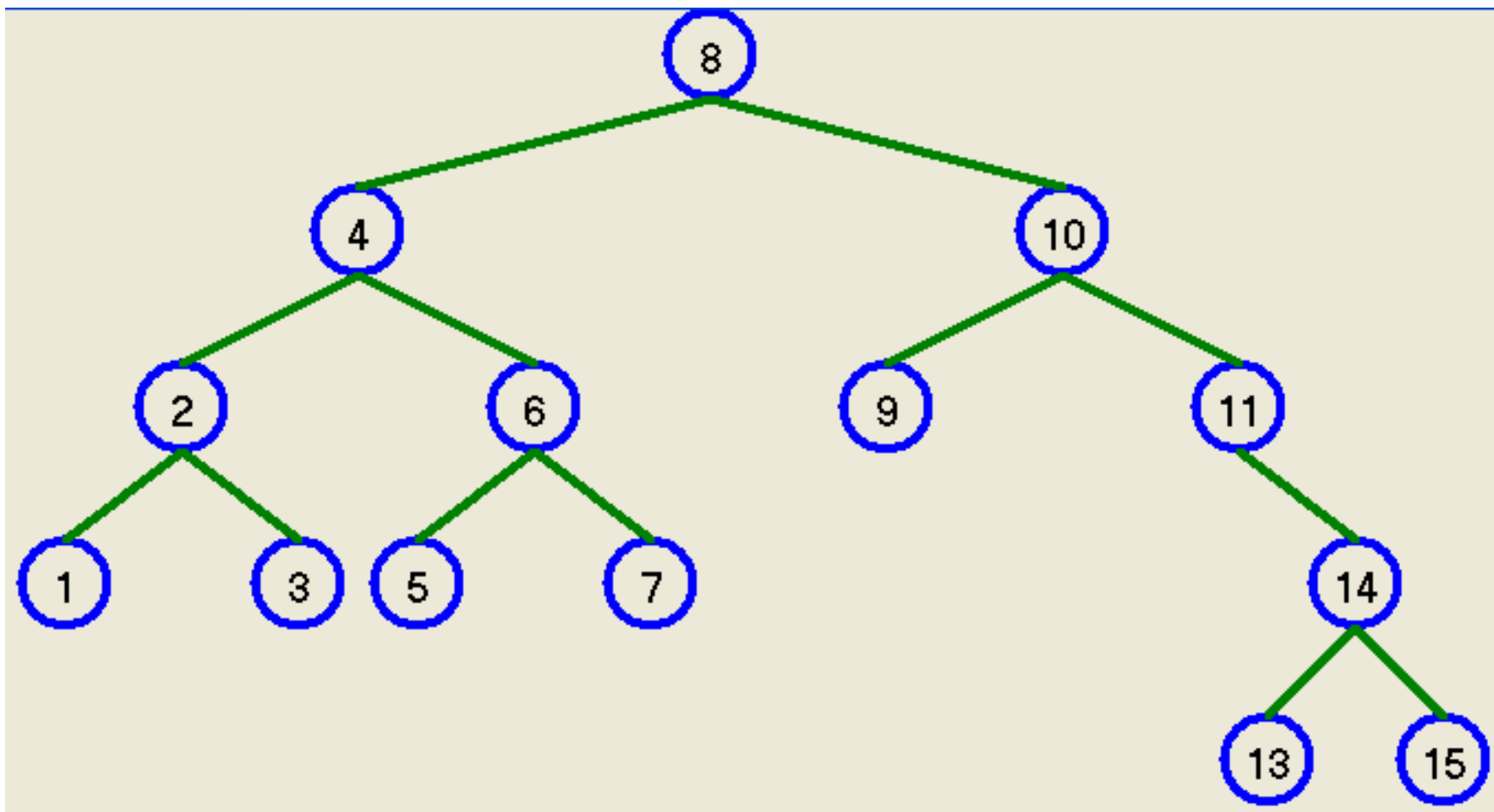
- ❑ O nó filho está à direita do pai.
- ❑ Ligar pai (dir) com filho (esq)
- ❑ Ligar o elemento mais à direita da esquerda do filho com o filho (dir).



NO NÃO TERMINAL: FILHO ESQ E DIR, PAI (ESQ)



NO NÃO TERMINAL: FILHO ESQ E DIR, PAI (ESQ)





REMOVER: DESVANTAGEM

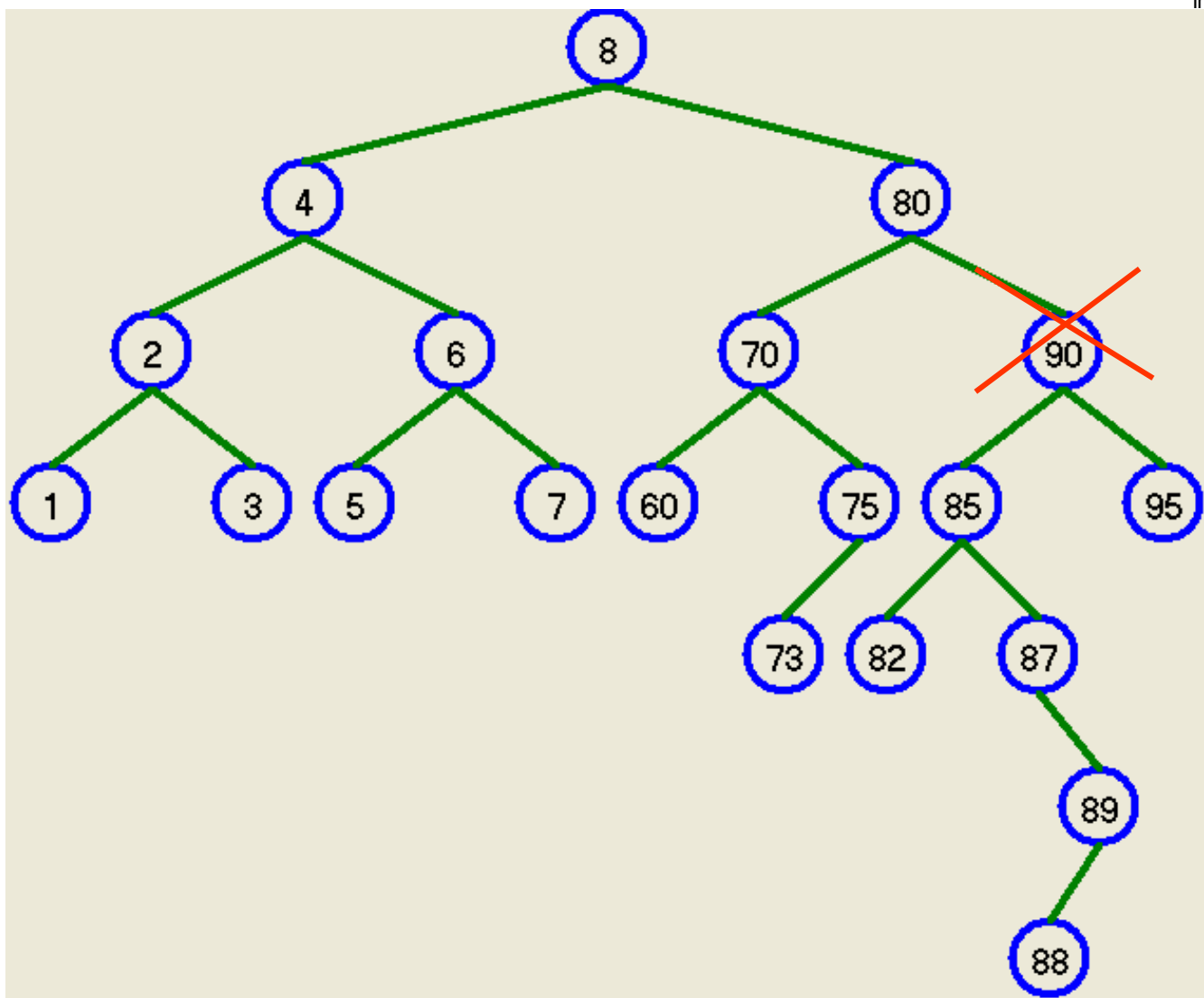
- ❑ A árvore degenera muito rapidamente.



REMOVER: POR SUBSTITUIÇÃO

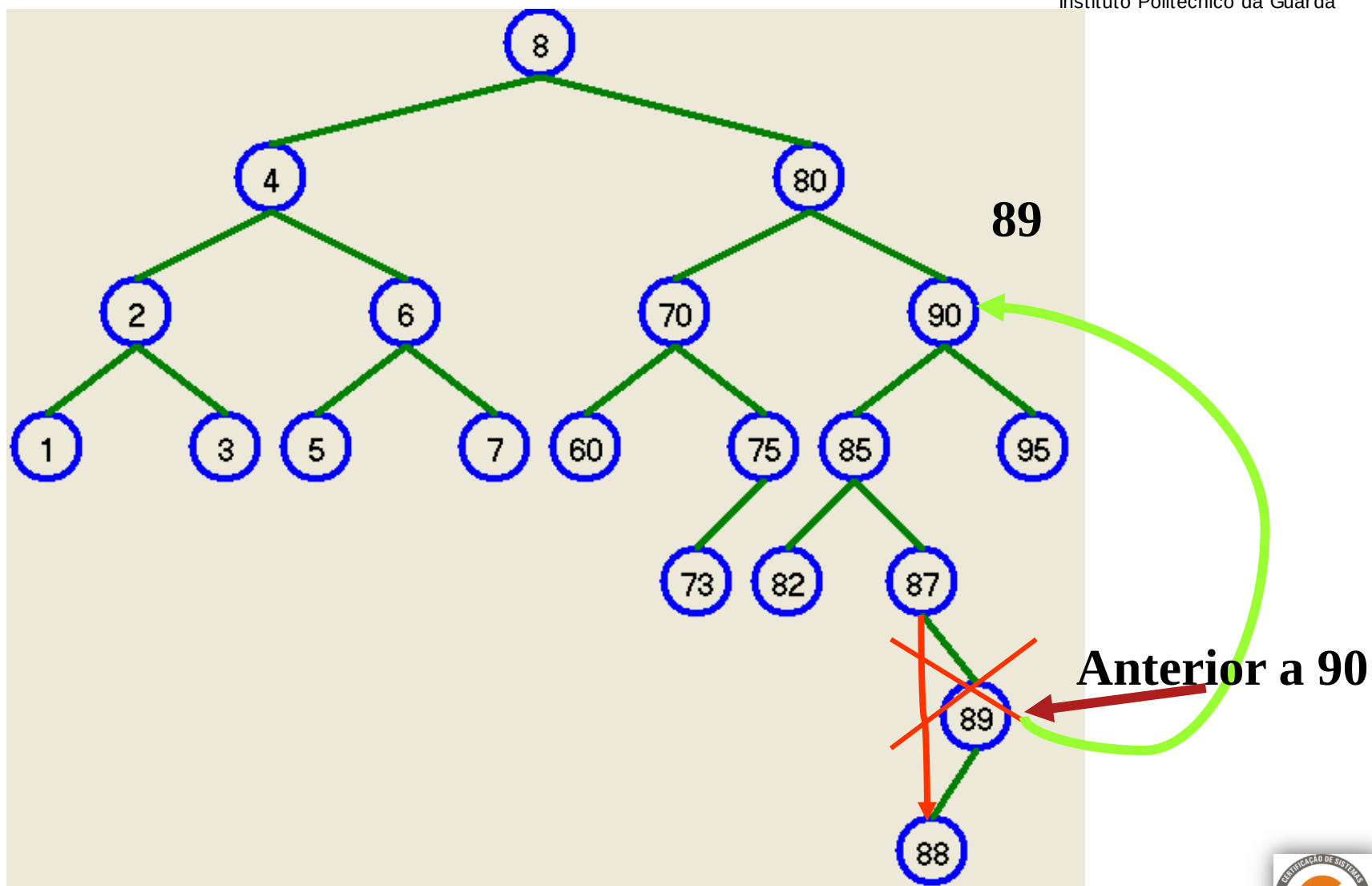
- ❑ Substitui-se o conteúdo do nó pelo conteúdo do nó imediatamente anterior em ordem simétrica e elimina-se este último, uma vez que este já não tem a subárvore direita.
- ❑ A árvore assim mantém-se binária de pesquisa.
- ❑ Efectivamente o nó imediatamente anterior em ordem simétrica, é o nó que se encontra na subárvore esquerda do nó a eliminar (todas chaves são menores), o mais à direita possível (dos menores é o maior)

EXEMPLO: REMOVE 80

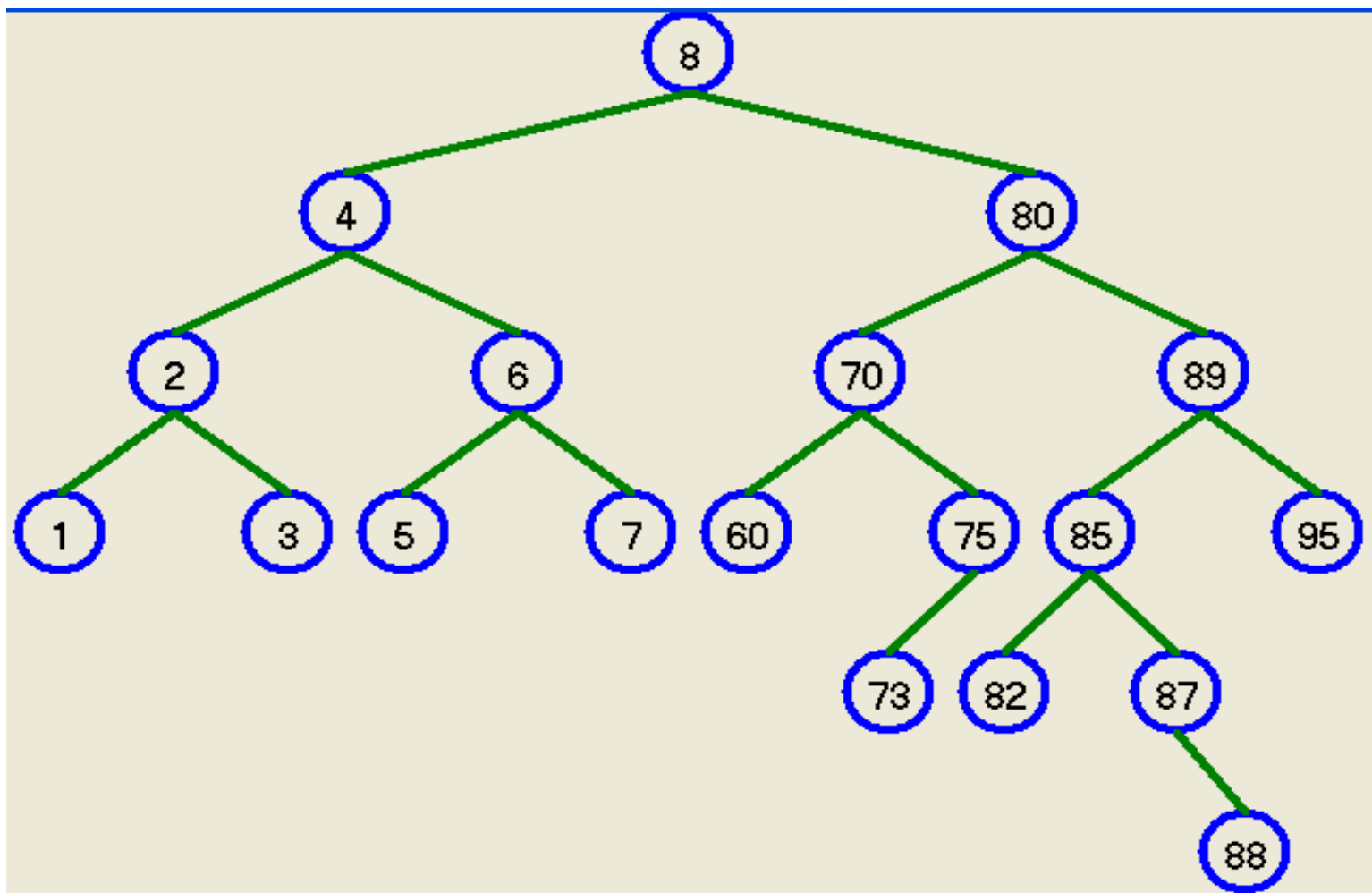




EXEMPLO: REMOVE 90



EXEMPLO: REMOVER 90





EXEMPLO: VANTAGEM

- A árvore tende a baixar de altura, porque os nós não terminais são substituídos por nós folhas ou por nós um nível acima das folhas.



IMPLEMENTAÇÃO: NÓ

```
class TNo {  
    private:  
        int valor;  
  
    public:  
        TNo *esq;  
        TNo *dir;  
        TNo() { esq = dir = 0; }  
        TNo(int v) {  
            esq = dir = 0;  
            valor = v;  
        }  
        void ColocaValor(int v) { valor = v; }  
        int TiraValor() { return valor; }  
};
```

IMPLEMENTAÇÃO: ÁRVORE

CLASS TARVORE



Escola Superior de Tecnologia e Gestão
Instituto Politécnico da Guarda

```
class TArvore {
    private:
        TNo *raiz;

    public:
        TArvore(int v){    raiz = new TNo(v);    }
        TArvore(){        raiz = 0;            }
        bool vazia() const {    return (raiz == 0);    }
        TNo *ObtemRaiz() {    return raiz;    }
        void PoeRaiz(TNo *r) {    raiz = r;    }
        bool EliminaSubstituicao(int e);
}
```

IMPLEMENTAÇÃO: TARVORE::BOOL INSERIRELEMENTO(INT E)



Escola Superior de Tecnologia e Gestão
Instituto Politécnico da Guarda

```
bool InserirElemento(int e) {
    TNo *pai=raiz, *filho = raiz;
    // - vazia: insere directamente novo dado
    if (vazia()) { raiz = new TNo(e); return 1; }
    // - não vazia: procura primeiro nó onde inserir novo dado
    while (filho != 0){    /* procura nó onde inserir dado */
        pai = filho;
        int x =filho->TiraValor();
        if (x == e) return 0;    /* já existe! */
        if (x > e) filho = filho->esq;
        else filho = filho->dir;
    }
    /* pai passa a nó intermédio e aponta para nova folha */
    TNo *folha = new TNo(e);
    if (pai->TiraValor() > e)
        pai->esq = folha;
    else
        pai->dir = folha;
    return 1;
}
```



IMPLEMENTAÇÃO:

TARVORE::BOOL ELIMINAR(INT E)

```
bool Eliminar(int e)
{
    TNo *aux = ObtemRaiz(); //inicia aux com a raiz
    TNo *pai = 0;
    while (aux != 0){
        if (aux->TiraValor() == e) break;
        pai = aux;
        if (e < aux->TiraValor()) // esquerda ou direita
            aux = aux->esq;
        else aux = aux->dir;
    }
    if (!aux )return 0;
    // verifica qual o caso - sem filho esquerdo ou direito
    if ( pai == 0){
        EliminarArvore();
        return 1;
    }
    ... continua ...
}
```

IMPLEMENTAÇÃO:

TARVORE::BOOL ELIMINAR(INT E)

```
... continuação ...  
if (aux == pai->esq) { // esq ou dir do pai ?  
    if (aux->dir == 0) {  
        pai->esq = aux->esq;  
        delete aux;  
    } else if (aux->esq == 0) { // muda pai e libera  
        pai->esq = aux->dir;  
        delete aux;  
    }  
    else { // um para esquerda e tudo à direita  
        TNo *mdir;  
        mdir = aux->esq;  
        // procura primeiro dprox NULL  
        while (mdir->dir != 0)  
            mdir = mdir->dir;  
        mdir->dir = aux->dir;  
        pai->esq = aux->esq;  
        delete aux;  
        return 1;  
    }  
}  
... continua ...
```

IMPLEMENTAÇÃO:

TARVORE::BOOL ELIMINAR(INT E)

... continuação ...

```
else {  
    if (aux->dir == 0) {  
        pai->dir = aux->esq;  
        delete aux;  
    } else if (aux->esq == 0)  
    {    // muda pai e libera  
        pai->dir = aux->dir;  
        delete aux;  
    } else { // um para esquerda e tudo à direita  
        TNo *mdir;  
        mdir = aux->esq;  
        while (mdir->dir != 0)  
            mdir = mdir->dir;  
        mdir->dir = aux->dir;  
        pai->dir = aux->esq;  
        delete aux;  
        return 1;  
    }  
}  
}
```




IMPLEMENTAÇÃO: TARVORE:: BOOL ELIMINASUBSTITUICAO(INT E)

```
bool EliminaSubstituicao(int e) {  
    TNo *pai=raiz, *filho = raiz, *no_sub = raiz, *avo;  
    while (no_sub != 0){    /* procura nó onde inserir dado */  
        pai = no_sub;  
        int x =no_sub->TiraValor();  
        if (x == e) {  
            avo = no_sub->esq;  
            pai = no_sub->esq;  
            filho = no_sub->esq;  
            while (filho){  
                avo = pai;  
                pai = filho;  
                filho = filho->dir;  
            }  
        }  
    }
```

... continua ...

IMPLEMENTAÇÃO: TARVORE:: BOOL ELIMINASUBSTITUICAO(INT E)

... continuação ...

```
no_sub->ColocaValor(pai->TiraValor());  
if (pai->esq) {  
    avo->dir = pai->esq;  
    delete pai;  
} else {  
    avo->dir = pai->dir;  
    delete pai;  
}  
return 1;  
}  
if (x > e) no_sub = no_sub->esq;  
else no_sub = no_sub->dir;  
}  
return 1;  
}
```

IMPLEMENTAÇÃO: BOOL EXISTE_E (INT E)



Escola Superior de Tecnologia e Gestão
Instituto Politécnico da Guarda

```
bool Existe_E (int e)
{
    TNo *N;
    N = Pesquisa_No_aux(ObtemRaiz(), e);
    if ((N != 0) && (N->TiraValor() == e))
        return 1;
    else
        return 0;
}
```



IMPLEMENTAÇÃO:

TNO *PESQUISA_NO(INT E)

```
TNo *Pesquisa_No(int e)
{
    return Pesquisa_No_aux(ObtemRaiz(), e);
}
```

```
TNo *Pesquisa_No_aux(TNo *r, int e)
{
    TNo *aux = r; //inicia aux com a raiz
    TNo *pai = r;
    while (aux != 0)
    {
        if (aux->TiraValor() == e) return pai;
        pai = aux;
        if (e < aux->TiraValor()) aux = aux->esq;
        else aux = aux->dir;
    }
    return pai;
}
```

IMPLEMENTAÇÃO:

VOID ELIMINARARVORE()

```
void EliminarArvore()  
{  
    EliminarArvoreAux(raiz);  
    raiz = 0;  
}
```

```
void EliminarArvoreAux(TNo *p)  
{  
    if (p != 0) {  
        EliminarArvoreAux(p->dir);  
        EliminarArvoreAux(p->esq);  
        delete p;  
    }  
}
```



- ❑ **Percorrendo a árvore "por níveis"**
- ❑ Os nós podem ser percorridos em uma quarta ordem, diferente da raiz-esquerda-direita, da esquerda-raiz-direita e da esquerda-direita-raiz. Para fazer isso, basta usar uma fila no lugar de uma pilha. (Veja programa 5.16, p.235, de Sedgewick.)
- ❑ // Imprime o item de cada nó de uma árvore binária h. // A função supõe que h != NULL.
- ❑ void imprime (link h) {
 - ❑ link *fila;
 - ❑ int i, f;
 - ❑ fila = malloc(count(h) * sizeof (link));
 - ❑ fila[0] = h; i = 0; f = 1;
 - ❑ while (f > i) {
 - ❑ h = fila[i++]; printf("%d\n", h->item);
 - ❑ if (h->l != NULL)
 - ❑ fila[f++] = h->l;
 - ❑ if (h->r != NULL)
 - ❑ fila[f++] = h->r;
 - ❑ }
 - ❑ free(fila); }

<http://www.ime.usp.br/~pf/mac0122-2002/aulas/trees.html>

- ❑ **Altura de um nó e altura de uma árvore**
- ❑ A **altura** (= *height*) de um nó h em uma árvore binária é a "distância" entre h e o seu descendente mais afastado. Mas precisamente, a altura de h é o número de links no mais longo caminho que leva de h até uma folha. Os caminhos a que essa definição se refere são os obtido pela iteração dos comandos $x = x \rightarrow l$ e $x = x \rightarrow r$, em qualquer ordem. Exemplo: a altura de uma folha é 0.
- ❑ A função abaixo (veja programa 5.17, p.236, do Sedgewick) calcula a altura de um nó h . Ela dá uma resposta razoável até mesmo quando h é NULL.
- ❑

```
// Devolve o altura de um nó h em uma árvore binária. int height(link h) { int u, v; if (h == NULL) return -1; u = height(h->l); v = height(h->r); if (u > v) return u+1; else return v+1; }
```

EXEMPLO

- ❑ Para ilustrar o conceito de árvore, eis uma pequena função (veja programa 5.17, p.236, do Sedgewick) que calcula o número de nós de uma árvore binária.
- ❑ // Esta função devolve o número de nós // da árvore binária cuja raiz é h.
- ❑ `int count(link h) {`
 - ❑ `if (h == NULL) return 0;`
 - ❑ `return count(h->l) + count(h->r) + 1;`
- ❑ `}`